

Contents

1 Misc	2	3.6 Segment Tree Li Chao Line	6	5.21 polygon / halfplane intersection	12	7.4 Floor Sum	19
1.1 Default Code	2	3.7 Segment Tree Li Chao Segment	7	5.22 polygon / minkowski sum	12	7.5 Josephus Problem	19
1.2 Run	2	3.8 Segment Tree Persistent	7	5.23 struct Cir	12	7.6 Lagrange any x	19
1.3 Diff	2	3.9 Segment Tree Zkw	7	5.24 Cir / Cir intersect	12	7.7 Lagrange continuous x	20
1.4 Hash Command	2	3.10 Sparse Table	8	5.25 Cir / point tangent	12	7.8 Linear Mod Inverse	20
1.5 Vimrc	2	3.11 Treap2	8	5.26 Cir / circle tangent	12	7.9 Lucas's Theorem	20
1.6 Custom Set PQ Sort	2	3.12 Trie	8	5.27 Cir / minimum enclosing circle	12	7.10 Matrix	20
1.7 Dynamic Bitset	2	4 Dynamic-Programming	9	5.28 Geometry Struct 3D	12	7.11 Matrix 01	21
1.8 Enumerate Subset	2	4.1 Digit DP	9	5.29 Pick's Theorem	13	7.12 Matrix Tree Theorem	21
1.9 Fast Input	2	4.2 Integer Partition	9	6 Graph	13	7.13 Miller Rabin	21
1.10 OEIS	2	4.3 Knapsack On Tree	9	6.1 SAT	13	7.14 Pollard Rho	21
1.11 OEIS	2	4.4 SOS DP	9	6.2 Augment Path	13	7.15 Polynomial	21
1.12 Pragma	3	5 Geometry	9	6.3 C3C4	14	7.16 Stirling's formula	22
1.13 Python	3	5.1 misc	9	6.4 Dinic	14	7.17 Theorem	22
1.14 Xor Basis	3	5.2 convex hull	9	6.5 Dominator Tree	14	7.18 josephus	22
1.15 disable ASLR	3	5.3 SVG writer	10	6.6 EdgeBCC	15	7.19 二元一次方程式	22
1.16 hash windows	3	5.4 struct point	10	6.7 EnumeratePlanarFace	15	7.20 數論分塊	22
1.17 random int	3	5.5 point / basic operation	10	6.8 Ford Fulkerson	16	7.21 最大質因數	22
2 Convolution	3	5.6 point / operators	10	6.9 HLD	16	7.22 歐拉公式	23
2.1 FFT	3	5.7 point / banana	10	6.10 Kosaraju SCC	16	7.23 歐拉定理	23
2.2 FFT any mod	4	5.8 point / rayHitSeg	10	6.11 Kuhn Munkres	17	7.24 錯排公式	23
2.3 FWT	4	5.9 point / else	10	6.12 LCA	17	8 String	23
2.4 Min Convolution Concave Concave	4	5.10 struct line	10	6.13 MCMF	17	8.1 AC automation	23
2.5 NTT mod 998244353	4	5.11 line / else	10	6.14 Tarjan SCC	18	8.2 Enumerate Runs	23
2.6 PolyInv	4	5.12 struct polygon	10	6.15 Theorem	18	8.3 Hash	23
2.7 PolySqrt	5	5.13 polygon / make convex hull	11	6.16 Tree Isomorphism	18	8.4 KMP	24
3 Data-Structure	5	5.14 polygon / in polygon	11	6.17 prufer reverse	18	8.5 Manacher	24
3.1 BIT	5	5.15 polygon / in convex	11	6.18 圖方樹	19	8.6 Min Rotation	24
3.2 Disjoint Set Persistent	5	5.16 polygon / cycle search	11	6.19 最大權閉合圖	19	8.7 Suffix Array	24
3.3 PBDS GP Hash Table	5	5.17 polygon / line cut convex	11	7 Math	19	8.8 Suffix Automata	25
3.4 PBDS Order Set	6	5.18 polygon / convex line intersect	11	7.1 Burnside's Lemma	19	8.9 Wildcard Matching	25
3.5 Segment Tree Add Set	6	5.19 polygon / segment pass convex interior	11	7.2 CRT	19	8.10 Z Algorithm	25
		5.20 polygon / convex tangent point	11	7.3 Catalan Number	19		

1 Misc

1.1 Default Code [24a798]

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 #define int long long
4 #define debug(a...) cerr << #a << " = ", dout(a)
5 void dout() { cerr << "\n"; }
6 template <typename A, typename... B>
7 void dout(A a, B... b) { cerr << a << ' ', dout(b...); }
8
9 void solve(){
10
11 }
12
13 signed main(){
14     ios::sync_with_stdio(0), cin.tie(0);
15
16     int t = 1;
17     while (t--){
18         solve();
19     }
20
21     return 0;
22 }
```

1.2 Run

```
1 from os import *
2 f = "pA"
3 while 1:
4     i = input("> ") or f; system("clear"); f = i
5     print(f"file = {f}")
6     if system(f"g++ {f}.cpp -std=c++17 -Wall -Wextra -Wshadow
7         -O2 -D LOCAL -g -fsanitize=undefined,address -o {f}
8         "):
9         print("CE"); continue
10    for x in sorted(listdir()):
11        if x.startswith(f) and x.endswith(".in"):
12            print(x); system(f"./{f} < {x}") and print("RE");
13            print()
```

1.3 Diff

```
1 set -e
2 g++ ac.cpp -o ac
3 g++ wa.cpp -o wa
4 for ((i=0;;i++))
5 do
6     echo "$i"
7     python3 gen.py > input
8     ./ac < input > ac.out
9     ./wa < input > wa.out
10    diff ac.out wa.out || break
11 done
```

1.4 Hash Command

```
1 cat file.cpp | cpp -dD -P -fpreprocessed | tr -d "[:space:]"
2 | md5sum | cut -c-6
```

1.5 Vimrc

```
1 se nu rnu bs=2 sw=4 ts=4 hls ls=2 si acd bo=all mouse=a
2
3 :inoremap " ""<Esc>i
4 :inoremap {<CR> {<CR><Esc>ko
5 :inoremap [{ {<Esc>i
6
7 ca hash w !cpp -dD -P -fpreprocessed \\\ tr -d "[:space:]" \\\
8 md5sum \\\ cut -c-6
9
10 " i+<esc>25A---+<esc>
11 " o/<esc>25A |<esc>
12 " ggVGyG35pGdd
```

1.6 Custom Set PQ Sort [d4df55]

```
1 // 所有自訂的結構體，務必檢查相等的 case，給所有元素一個排序
2 // 的依據
3 struct my_struct{
4     int val;
5     my_struct(int _val) : val(_val) {}
6 };
7
8 auto cmp = [](my_struct a, my_struct b) {
9     return a.val > b.val;
10 };
11
12 set<my_struct, decltype(cmp)> ss({1, 2, 3}, cmp);
13 priority_queue<my_struct, vector<my_struct>, decltype(cmp)>
14     pq(cmp, {1, 2, 3});
15 map<my_struct, my_struct, decltype(cmp)> mp({{1, 4}, {2, 5},
16     {3, 6}}, cmp);
```

1.7 Dynamic Bitset [c78aa8]

```
1 const int MAXN = 2e5 + 5;
2 template <int len = 1>
3 void solve(int n) {
4     if (n > len) {
5         solve<min(len*2, MAXN)>(n);
6         return;
7     }
8     bitset<len> a;
9 }
```

1.8 Enumerate Subset [a13e46]

```
1 // 時間複雜度 O(3^n)
2 // 枚舉每個 mask 的子集
3 for (int mask=0; mask<(1<n); mask++){
4     for (int s=mask; s>=0; s=(s-1)&m){
5         // s 是 mask 的子集
6         if (s==0) break;
7     }
8 }
```

1.9 Fast Input [6f8879]

```
1 // fast IO
2 // 6f8879
3 inline char readchar(){
```

```
4     static char buffer[BUFSIZ], *now = buffer + BUFSIZ, *
5     end = buffer + BUFSIZ;
6     if (now == end)
7     {
8         if (end < buffer + BUFSIZ)
9             return EOF;
10        end = (buffer + fread(buffer, 1, BUFSIZ, stdin));
11        now = buffer;
12    }
13    return *now++;
14 }
15 inline int nextint(){
16     int x = 0, c = readchar(), neg = false;
17     while(('0' > c || c > '9') && c!='-' && c!=EOF) c =
18         readchar();
19     if(c == '-') neg = true, c = readchar();
20     while('0' <= c && c <= '9') x = (x<<3) + (x<<1) + (c^'0')
21         , c = readchar();
22     if(neg) x = -x;
23     return x; // returns 0 if EOF
```

1.10 OEIS [25cf74]

```
1 // 若一個線性遞迴有 k 項，給他恰好 2*k 個項可以求出線性遞迴
2 // s = 1 1 3 7 17，則它會回傳 1 2
3 vector<int> BerlekampMassey(vector<int> s) {
4     if (s.empty()) return vector<int>();
5     int n = s.size(), L = 0, m = 0;
6     vector<int> C(n), B(n), T;
7     C[0] = B[0] = 1;
8
9     int b = 1;
10    for (int i=0; i<n; i++) {
11        ++m; int d = s[i]%MOD;
12        for (int j=1; j<L+1; j++) d = (d+C[j]*s[i-j])%MOD;
13        if (!d) continue;
14        T = C; int coef = d*qp(b, MOD-2, MOD)%MOD;
15        for (int j=m; j<n; j++) C[j] = (C[j]-coef*B[j-m])%
16            MOD;
17        if (2*L>i) continue;
18        L = i+1-L; B=T; b=d; m=0;
19    }
20    C.resize(L+1); C.erase(C.begin());
21    for (int &x : C) x = (MOD-x)%MOD;
22    reverse(C.begin(), C.end());
23    return C;
```

1.11 OEIS

```
1 from fractions import Fraction as F
2 def BM(s):
3     n = len(s); L=m=0; C=[F(0)]*n; B=[F(0)]*n; C[0]=B[0]=F(1)
4     ; b=F(1)
5     for i in range(n):
6         m+=1; d=sum(C[j]*s[i-j] for j in range(L+1))
7         if d==0: continue
8         T = C[:]; coef = F(d/b)
9         for j in range(m, n): C[j] -= coef*B[j-m]
10        if 2*L<=i: L, B, b, m = i+1-L, T, d, 0
11    if len(C)<L+1: C += [F(0)] * (L+1-len(C))
12    return [-x for x in C[1:L+1]]
```

1.12 Pragma [09d13e]

```
1 #pragma GCC optimize("O3,unroll-loops")
2 #pragma GCC target("avx,avx2,sse,sse2,sse3,sse4,popcnt")
```

1.13 Python

```
1 # Decimal
2 from decimal import *
3 getcontext().prec = 6
4
5 # system setting
6 sys.setrecursionlimit(100000)
7 sys.set_int_max_str_digits(10000)
8
9 # turtle
10 from turtle import *
11
12 N = 3000000010
13 setworldcoordinates(-N, -N, N, N)
14 hideturtle()
15 speed(100)
16
17 def draw_line(a, b, c, d):
18     teleport(a, b)
19     goto(c, d)
20
21 def write_dot(x, y, text, diff=1): # diff = 文字的偏移
22     teleport(x, y)
23     dot(5, "red")
24
25     teleport(x+N/100*diff, y+N/100*diff)
26     write(text, font=("Arial", 5, "bold"))
27
28 # usage
29 draw_line(*a[i], *(a[i-1]))
30 write_dot(*a[i], str(a[i]))
31 # 以自己左方 100 單位為圓心向前畫一個 90 度的弧 / 正五邊形
   ( 參數可以是負的 )
32 circle(100, extent = 90)
33 circle(100, steps = 5)
34 left(90) # 左轉 90 度
35 fd(-100) # 倒退 100 單位
36 penup()
37 pendown()
38
39 tracer(0, 0) # IO 優化 · 必須在程式碼最後手動 update()
40 pos() # 回傳目前座標
41 heading() # 回傳目前方向：0 是右方、90 是上方
42 color("red") # 改變顏色 · 可以用文字或是 #000000 的格式
43
44 # OOP
45 class Point:
46     def __init__(self, x, y):
47         self.x = x
48         self.y = y
49
50     def __add__(self, o): # use dir(int) to know operator
       name
51         return Point(self.x+o.x, self.y+o.y)
52
53 @property
54 def distance(self):
55     return (self.x**2 + self.y**2)**(0.5)
```

```
56
57 a = Point(3, 4)
58 print(a.distance)
59
60 # Fraction
61 from fractions import Fraction
62 a = Fraction(Decimal(1.1))
63 a.numerator # 分子
64 a.denominator # 分母
```

1.14 Xor Basis [123f2d]

```
1 vector<int> basis;
2 void add_vector(int x){
3     for (auto v : basis){
4         x=min(x, x^v);
5     }
6     if (x) basis.push_back(x);
7 }
8
9 // 給一數字集合 S · 求能不能 XOR 出 x
10 bool check(int x){
11     for (auto v : basis){
12         x=min(x, x^v);
13     }
14     return (x == 0);
15 }
16
17 // 給一數字集合 S · 求能 XOR 出多少數字
18 // 答案等於 2^{basis 的大小}
19
20 // 給一數字集合 S · 求 XOR 出最大的數字
21 int get_max(){
22     int ans=0;
23     for (auto v : basis){
24         ans=max(ans, ans^v);
25     }
26     return ans;
27 }
```

1.15 disable ASLR [29a49f]

```
1 // Disable randomization of memory addresses
2 // setarch $(uname -m) -R /bin/bash
3
4 #include <sys/personality.h>
5 #include <bits/stdc++.h>
6 int main(int argc, char* argv[]){
7     personality(ADDR_NO_RANDOMIZE);
8     execv(argv[1], argv+1);
9 }
```

1.16 hash windows

```
1 def get_hash(path):
2     from subprocess import run, PIPE
3     from hashlib import md5
4
5     p = run(
6         ["cpp", "-dD", "-P", "-fpreprocessed", path],
7         stdout = PIPE,
8         stderr = PIPE,
9         text = True
```

```
10     )
11
12     if p.returncode != 0:
13         raise RuntimeError(p.stderr)
14
15     s = ''.join(p.stdout.split())
16     ret = md5(s.encode()).hexdigest()
17     return ret[:6]
18
19 print(get_hash("Suffix_Array.cpp"))
```

1.17 random int [9cc603]

```
1 mt19937 seed(chrono::steady_clock::now().time_since_epoch().
   count());
2 int rng(int l, int r){
3     return uniform_int_distribution<int>(l, r)(seed);
4 }
```

2 Convolution

2.1 FFT [16591a]

```
1 typedef complex<double> cd;
2
3 // 778272
4 void FFT(vector<cd> &a) {
5     int n = a.size(), L = 31-__builtin_clz(n);
6     vector<complex<long double>> R(2, 1);
7     vector<cd> rt(2, 1);
8     for (int k=2 ; k<n ; k*=2){
9         R.resize(n), rt.resize(n);
10        auto x = polar(1.0L, acos(-1.0L) / k);
11        for (int i=k ; i<2*k ; i++) rt[i] = R[i] = (i&1 ? R[i
12            /2]*x : R[i/2]);
13    }
14
15    vector<int> rev(n);
16    for (int i=0 ; i<n ; i++) rev[i] = (rev[i/2] | (i&1)<<L)
17        /2;
18    for (int i=0 ; i<n ; i++) if (i<rev[i]) swap(a[i], a[rev[
19        i]]);
20    for (int k=1 ; k<n ; k*=2){
21        for (int i=0 ; i<n ; i+=2*k){
22            for (int j=0 ; j<k ; j++){
23                // cd z = rt[j+k] * a[i+j+k];
24                auto x = (double *)&rt[j+k], y = (double *)&a
25                    [i+j+k];
26                cd z(x[0]*y[0] - x[1]*y[1], x[0]*y[1] + x[1]*
27                    y[0]);
28                a[i+j+k] = a[i+j]-z;
29                a[i+j] += z;
30            }
31        }
32    }
33 }
34
35 // 192528
36 vector<double> PolyMul(const vector<double> a, const vector<
   double> b){
37     if (a.empty() || b.empty()) return {};
38     vector<double> res(a.size()+b.size()-1);
39     int L = 32 - __builtin_clz(res.size()), n = 1<<L;
40     vector<cd> in(n), out(n);
41     copy(a.begin(), a.end(), in.begin());
```

```

37 for (int i=0 ; i<b.size() ; i++) in[i].imag(b[i]);
38 FFT(in);
39 for (cd& x : in) x *= x;
40 for (int i=0 ; i<n ; i++) out[i] = in[-i & (n - 1)] -
    conj(in[i]);
41 FFT(out);
42 for (int i=0 ; i<res.size() ; i++) res[i] = imag(out[i])
    / (4 * n);
43 return res;
44 }

```

2.2 FFT any mod [412000]

```

1 // n=524288, ~400ms
2 const int MOD = 998244353;
3 typedef complex<double> cd;
4
5 // bb6d94
6 void FFT(vector<cd> &a) {
7     int n = a.size(), L = 31-__builtin_clz(n);
8     vector<complex<long double>> R(2, 1);
9     vector<cd> rt(2, 1);
10    for (int k=2 ; k<n ; k*=2){
11        R.resize(n);
12        rt.resize(n);
13        auto x = polar(1.0L, acos(-1.0L) / k);
14        for (int i=k ; i<2*k ; i++){
15            rt[i] = R[i] = (i&1 ? R[i/2]*x : R[i/2]);
16        }
17    }
18
19    vector<int> rev(n);
20    for (int i=0 ; i<n ; i++) rev[i] = (rev[i/2] | (i&1)<<L)
        /2;
21    for (int i=0 ; i<n ; i++) if (i<rev[i]) swap(a[i], a[rev[i]]);
22    for (int k=1 ; k<n ; k*=2){
23        for (int i=0 ; i<n ; i+=2*k){
24            for (int j=0 ; j<k ; j++){
25                auto x = (double *)&rt[j+k];
26                auto y = (double *)&a[i+j+k];
27                cd z(x[0]*y[0] - x[1]*y[1], x[0]*y[1] + x[1]*
                    y[0]);
28                a[i+j+k] = a[i+j]-z;
29                a[i+j] += z;
30            }
31        }
32    }
33 }
34
35 // c3dcf6
36 vector<int> PolyMul(vector<int> a, vector<int> b){
37     if (a.empty() || b.empty()) return {};
38     vector<int> res(a.size()+b.size()-1);
39     int B = 32-__builtin_clz(res.size()), n = (1<<B), cut = (
        int)(sqrt(MOD));
40     vector<cd> L(n), R(n), outs(n), outl(n);
41
42     for (int i=0 ; i<a.size() ; i++) L[i] = cd((int) a[i]/cut
        , (int)a[i]%cut);
43     for (int i=0 ; i<b.size() ; i++) R[i] = cd((int) b[i]/cut
        , (int)b[i]%cut);
44     FFT(L), FFT(R);
45     for (int i=0 ; i<n ; i++){
46         int j = -i&(n-1);

```

```

47         outl[j] = (L[i]+conj(L[j])) * R[i]/(2.0*n);
48         outs[j] = (L[i]-conj(L[j])) * R[i]/(2.0*n)/1i;
49     }
50     FFT(outl), FFT(outs);
51     for (int i=0 ; i<res.size() ; i++){
52         int av = (int)(real(outl[i])+0.5), cv = (int)(imag(
            outs[i])+0.5);
53         int bv = (int)(imag(outl[i])+0.5) + (int)(real(outs[i]
            ))+0.5);
54         res[i] = ((av%MOD*cut+bv) % MOD*cut+cv) % MOD;
55     }
56     return res;
57 }

```

2.3 FWT [e50788]

```

1 // 已經把 mint 刪掉，需要增加註解
2 vector<int> xor_convolution(vector<int> a, vector<int> b, int
    k) {
3     if (k==0) return vector<int>{a[0]*b[0]};
4     vector<int> aa(1 << (k - 1)), bb(1 << (k - 1));
5     for (int i=0 ; i<(1<<(k-1)) ; i++){
6         aa[i] = a[i] + a[i + (1 << (k - 1))];
7         bb[i] = b[i] + b[i + (1 << (k - 1))];
8     }
9     vector<int> X = xor_convolution(aa, bb, k - 1);
10    for (int i=0 ; i<(1<<(k-1)) ; i++){
11        aa[i] = a[i] - a[i + (1 << (k - 1))];
12        bb[i] = b[i] - b[i + (1 << (k - 1))];
13    }
14    vector<int> Y = xor_convolution(aa, bb, k - 1);
15    vector<int> c(1 << k);
16    for (int i=0 ; i<(1<<(k-1)) ; i++){
17        c[i] = (X[i] + Y[i]) / 2;
18        c[i + (1 << (k - 1))] = (X[i] - Y[i]) / 2;
19    }
20    return c;
21 };

```

2.4 Min Convolution Concave Concave [ffb28d]

```

1 // 需要增加註解
2 // min convolution
3 vector<int> mkk(vector<int> a, vector<int> b) {
4     vector<int> slope;
5     FOR (i, 1, ssize(a) - 1) slope.pb(a[i] - a[i - 1]);
6     FOR (i, 1, ssize(b) - 1) slope.pb(b[i] - b[i - 1]);
7     sort(all(slope));
8     slope.insert(begin(slope), a[0] + b[0]);
9     partial_sum(all(slope), begin(slope));
10    return slope;
11 }

```

2.5 NTT mod 998244353 [976c8d]

```

1 const int MOD = (119 << 23) + 1, ROOT = 62; // = 998244353
2 // For p < 2^30 there is also e.g. 5 << 25, 7 << 26, 479 <<
    21
3 // and 483 << 21 (same root). The last two are > 10^9.
4
5 // 7e041b
6 vector<int> rt(2, 1);
7 void NTT_init(int n){
8     static bool lastInit = 0;

```

```

9     if (lastInit<n){
10         lastInit = n;
11         for (int k=2, s=2 ; k<n ; k*=2, s++){
12             rt.resize(n);
13             int z[] = {1, qp(ROOT, MOD>>s, MOD)};
14             for (int i=k ; i<2*k ; i++) rt[i] = rt[i/2]*z[i
                &1]%MOD;
15         }
16     }
17 }
18
19 // fec4b8
20 void NTT(vector<int> &a, int isInverse = false) {
21     int n = a.size();
22     int L = 31-__builtin_clz(n);
23     NTT_init(n);
24
25     vector<int> rev(n);
26     for (int i=0 ; i<n ; i++) rev[i] = (rev[i/2] | (i&1)<<L)/2;
27     for (int i=0 ; i<n ; i++){
28         if (i<rev[i]) swap(a[i], a[rev[i]]);
29     }
30
31     for (int k=1 ; k<n ; k*=2){
32         for (int i=0 ; i<n ; i+=2*k){
33             for (int j=0 ; j<k ; j++){
34                 int z = rt[j+k]*a[i+j+k]%MOD, &ai = a[i+j];
35                 a[i+j+k] = ai-z+(z>ai ? MOD : 0);
36                 ai += (ai+z>MOD ? z-MOD : z);
37             }
38         }
39     }
40
41     if (isInverse) {
42         reverse(a.begin()+1, a.end());
43         int n_inv = qp(n, MOD-2, MOD);
44         for (int i=0 ; i<n ; i++) a[i] = a[i]*n_inv%MOD;
45     }
46 }
47
48 // 771576
49 vector<int> polyMul(vector<int> a, vector<int> b){
50     if (a.empty() || b.empty()) return {};
51     int s = a.size()+b.size()-1, B = 32-__builtin_clz(s), n =
        1<<B;
52     vector<int> L(a), R(b), out(n);
53     L.resize(n), R.resize(n);
54     NTT(L), NTT(R);
55     for (int i=0 ; i<n ; i++) out[i] = L[i]*R[i]%MOD;
56     NTT(out, true);
57     out.resize(s);
58     return out;
59 }

```

2.6 PolyInv [b388fb]

```

1 vector<int> polyInv(vector<int> a){
2     int n = a.size();
3     vector<int> ret(1, qp(a[0], MOD-2, MOD));
4
5     for (int m=1 ; m<n ; m<=1){
6         if (n<2*m) a.resize(2*m);
7         vector<int> v1(a.begin(), a.begin()+2*m), v2 = ret;
8         v1.resize(4*m), v2.resize(4*m);
9         NTT(v1), NTT(v2);

```

```

10     for (int i=0; i<4*m; i++) v1[i] = v1[i]*v2[i]%MOD*
        v2[i]%MOD;
11     NTT(v1, true);
12     ret.resize(2*m);
13     for (int i=0; i<m; i++) ret[i] = (ret[i]+ret[i])%
        MOD;
14     for (int i=0; i<2*m; i++) ret[i] = ((ret[i]-v1[i])%
        MOD+MOD)%MOD;
15 }
16 ret.resize(n);
17 return ret;
18 }

```

2.7 PolySqrt [1ea321]

```

1 // ---- sqrt for formal power series over MOD=998244353 ----
2 int mod_sqrt(int a) { // Tonelli-Shanks; return -1 if no
    solution
3     a %= MOD; if (a < 0) a += MOD;
4     if (a == 0) return 0;
5     if (qp(a, (MOD - 1) / 2, MOD) == MOD - 1) return -1; //
        non-residue
6     if (MOD % 4 == 3) return qp(a, (MOD + 1) / 4, MOD);
7     int q = MOD - 1, s = 0;
8     while ((q & 1) == 0) q >>= 1, ++s;
9     int z = 2;
10    while (qp(z, (MOD - 1) / 2, MOD) != MOD - 1) ++z; // find
        non-residue
11    long long c = qp(z, q, MOD);
12    long long x = qp(a, (q + 1) / 2, MOD);
13    long long t = qp(a, q, MOD);
14    int m = s;
15    while (t != 1) {
16        int i = 1;
17        long long tt = t * t % MOD;
18        while (tt != 1) {
19            tt = tt * tt % MOD;
20            ++i;
21            if (i == m) return -1; // shouldn't happen
22        }
23        long long b = qp(c, 1LL << (m - i - 1), MOD);
24        x = x * b % MOD;
25        long long b2 = b * b % MOD;
26        t = t * b2 % MOD;
27        c = b2;
28        m = i;
29    }
30    return (int)x;
31 }
32
33 vector<int> polySqrt(vector<int> a) {
34     int n = (int)a.size();
35
36     // all zero -> zero
37     bool all0 = true;
38     for (int x : a) if (x % MOD != 0) { all0 = false; break;
39     }
40     if (all0) return vector<int>(n, 0);
41
42     // first nonzero position
43     int k = 0;
44     while (k < n && a[k] == 0) ++k;
45     if (k & 1) return {}; // odd -> impossible
46     int shift = k / 2;

```

```

47 // strip x^k
48 vector<int> f;
49 f.reserve(n - k);
50 for (int i = k; i < n; ++i) f.push_back(a[i]);
51
52 // constant term sqrt, choose canonical sign
53 int c0 = (f.empty() ? 0 : f[0] % MOD);
54 int s0 = mod_sqrt(c0);
55 if (s0 == -1) return {};
56 if (s0 > MOD - s0) s0 = MOD - s0; // choose the smaller
        nonnegative root
57
58 // We must produce N terms after placing the shift =>
        need = N - shift
59 int need = n - shift;
60 if ((int)f.size() < need) f.resize(need, 0);
61
62 // Newton: g_{new} = (g + f/g) / 2 (mod x^m)
63 const int INV2 = (MOD + 1) / 2;
64 vector<int> g(1, s0);
65
66 for (int m = 1; m < need; m <= 1) {
67     int lim = min(need, m <= 1);
68
69     vector<int> g_cut = g; g_cut.resize(lim);
70     vector<int> inv_g = polyInv(g_cut); // inv up
        to lim
71     vector<int> f_cut(f.begin(), f.begin() + lim);
72
73     vector<int> q = polyMul(inv_g, f_cut);
74     if ((int)q.size() > lim) q.resize(lim);
75
76     g.resize(lim);
77     for (int i = 0; i < lim; ++i) {
78         long long gi = (i < (int)g_cut.size() ? g_cut[i]
79             : 0);
80         long long qi = (i < (int)q.size() ? q[i] : 0);
81         g[i] = (gi + qi) % MOD * 1LL * INV2 % MOD;
82     }
83     g.resize(need);
84
85     // place back the x^{k/2} shift
86     vector<int> res(n, 0);
87     for (int i = 0; i < need && i + shift < n; ++i) res[i +
        shift] = g[i];
88     return res;
89 }

```

3 Data-Structure

3.1 BIT [7ef3a9]

```

1 vector<int> BIT(MAX_SIZE);
2
3 // const int MAX_N = (1<<20)
4 int k_th(int k){ // 回傳 BIT 中第 k 小的元素 (based-1)
5     int res = 0;
6     for (int i=MAX_N>>1; i>=1; i>=1)
7         if (BIT[res+i]<k)
8             k -= BIT[res+i];
9     return res+1;
10 }

```

3.2 Disjoint Set Persistent [447002]

```

1 struct Persistent_Disjoint_Set{
2     Persistent_Segment_Tree arr, sz;
3
4     void init(int n){
5         arr.init(n);
6         vector<int> v1;
7         for (int i=0; i<n; i++){
8             v1.push_back(i);
9         }
10        arr.build(v1, 0);
11
12        sz.init(n);
13        vector<int> v2;
14        for (int i=0; i<n; i++){
15            v2.push_back(1);
16        }
17        sz.build(v2, 0);
18    }
19
20    int find(int a){
21        int res = arr.query_version(a, a+1, arr.version.size()
22            -1).val;
23        if (res==a) return a;
24        return find(res);
25    }
26
27    bool unite(int a, int b){
28        a = find(a);
29        b = find(b);
30
31        if (a!=b){
32            int sz1 = sz.query_version(a, a+1, arr.version.
33                size()-1).val;
34            int sz2 = sz.query_version(b, b+1, arr.version.
35                size()-1).val;
36
37            if (sz1<sz2){
38                arr.update_version(a, b, arr.version.size()
39                    -1);
40                sz.update_version(b, sz1+sz2, arr.version.
41                    size()-1);
42            }else{
43                arr.update_version(b, a, arr.version.size()
44                    -1);
45                sz.update_version(a, sz1+sz2, arr.version.
46                    size()-1);
47            }
48            return true;
49        }
50        return false;
51    }
52 }

```

3.3 PBDS GP Hash Table [866cf6]

```

1 #include <ext/pb_ds/assoc_container.hpp>
2 using namespace __gnu_pbds;
3 typedef tree<int, null_type, less<int>, rb_tree_tag,
4     tree_order_statistics_node_update> order_set;
5 struct custom_hash {
6     static uint64_t splitmix64(uint64_t x) {
7         // http://xorshift.di.unimi.it/splitmix64.c
8     }
9 }

```

```

7   x += 0x9e3779b97f4a7c15;
8   x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
9   x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
10  return x ^ (x >> 31);
11 }
12
13 size_t operator()(uint64_t x) const {
14     static const uint64_t FIXED_RANDOM = chrono::
15         steady_clock::now().time_since_epoch().count();
16     return splitmix64(x + FIXED_RANDOM);
17 };
18
19 gp_hash_table<int, int, custom_hash> ss;

```

3.4 PBDS Order Set [231774]

```

1 // .find_by_order(k) 回傳第 k 小的值 (based-0)
2 // .order_of_key(k) 回傳有多少元素比 k 小
3 // 不能在 #define int long long 後 #include 檔案
4 #include <ext/pb_ds/assoc_container.hpp>
5 #include <ext/pb_ds/tree_policy.hpp>
6 using namespace __gnu_pbds;
7 typedef tree<int, null_type, less<int>, rb_tree_tag,
8     tree_order_statistics_node_update> order_set;

```

3.5 Segment Tree Add Set [bb1898]

```

1 // [ll, rr), based-0
2 // 使用前記得 init(陣列大小), build(陣列名稱)
3 // add(ll, rr): 區間修改
4 // set(ll, rr): 區間賦值
5 // query(ll, rr): 區間求和 / 求最大值
6 struct SegmentTree{
7     struct node{
8         int add_tag = 0;
9         int set_tag = 0;
10        int sum = 0;
11        int ma = 0;
12    };
13
14    vector<node> arr;
15
16    SegmentTree(int n){
17        arr.resize(n<<2);
18    }
19
20    node pull(node A, node B){
21        node C;
22        C.sum = A.sum+B.sum;
23        C.ma = max(A.ma, B.ma);
24        return C;
25    }
26
27    // cce0c8
28    void push(int idx, int ll, int rr){
29        if (arr[idx].set_tag!=0){
30            arr[idx].sum = (rr-ll)*arr[idx].set_tag;
31            arr[idx].ma = arr[idx].set_tag;
32            if (rr-ll>1){
33                arr[idx*2+1].add_tag = 0;
34                arr[idx*2+1].set_tag = arr[idx].set_tag;
35                arr[idx*2+2].add_tag = 0;
36                arr[idx*2+2].set_tag = arr[idx].set_tag;

```

```

37        }
38        arr[idx].set_tag = 0;
39    }
40    if (arr[idx].add_tag!=0){
41        arr[idx].sum += (rr-ll)*arr[idx].add_tag;
42        arr[idx].ma += arr[idx].add_tag;
43        if (rr-ll>1){
44            arr[idx*2+1].add_tag += arr[idx].add_tag;
45            arr[idx*2+2].add_tag += arr[idx].add_tag;
46        }
47        arr[idx].add_tag = 0;
48    }
49 }
50
51 void build(vector<int> &v, int idx = 0, int ll = 0, int
52     rr = n){
53     if (rr-ll==1){
54         arr[idx].sum = v[ll];
55         arr[idx].ma = v[ll];
56     }else{
57         int mid = (ll+rr)/2;
58         build(v, idx*2+1, ll, mid);
59         build(v, idx*2+2, mid, rr);
60         arr[idx] = pull(arr[idx*2+1], arr[idx*2+2]);
61     }
62 }
63
64 void add(int ql, int qr, int val, int idx = 0, int ll =
65     0, int rr =n){
66     push(idx, ll, rr);
67     if (rr<=ql || qr<=ll) return;
68     if (ql<=ll && rr<=qr){
69         arr[idx].add_tag += val;
70         push(idx, ll, rr);
71         return;
72     }
73     int mid = (ll+rr)/2;
74     add(ql, qr, val, idx*2+1, ll, mid);
75     add(ql, qr, val, idx*2+2, mid, rr);
76     arr[idx]=pull(arr[idx*2+1], arr[idx*2+2]);
77 }
78
79 void set(int ql, int qr, int val, int idx=0, int ll=0,
80     int rr=n){
81     push(idx, ll, rr);
82     if (rr<=ql || qr<=ll) return;
83     if (ql<=ll && rr<=qr){
84         arr[idx].add_tag = 0;
85         arr[idx].set_tag = val;
86         push(idx, ll, rr);
87         return;
88     }
89     int mid = (ll+rr)/2;
90     set(ql, qr, val, idx*2+1, ll, mid);
91     set(ql, qr, val, idx*2+2, mid, rr);
92     arr[idx] = pull(arr[idx*2+1], arr[idx*2+2]);
93 }
94
95 node query(int ql, int qr, int idx = 0, int ll = 0, int
96     rr = n){
97     push(idx, ll, rr);
98     if (rr<=ql || qr<=ll) return node();
99     if (ql<=ll && rr<=qr) return arr[idx];
100
101     int mid = (ll+rr)/2;

```

```

98     return pull(query(ql, qr, idx*2+1, ll, mid), query(ql
99         , qr, idx*2+2, mid, rr));
100 } ST;

```

3.6 Segment Tree Li Chao Line [45b8ba]

```

1 // 全部都是 0-based
2 // 宣告: LC_Segment_Tree st(n);
3 // 函式:
4 // update({a, b}): 插入一條 y=ax+b 的全域直線
5 // query(x): 查詢所有直線在位置 x 的最小值
6 const int MAX_V = 1e6+10; // 值域最大值
7
8 struct LC_Segment_Tree{
9     struct Node{ // y = ax+b
10         int a = 0;
11         int b = INF;
12
13         int y(int x){
14             return a*x+b;
15         }
16     };
17     vector<Node> arr;
18
19     LC_Segment_Tree(int n = 0){
20         arr.resize(4*n);
21     }
22
23     void update(Node val, int idx = 0, int ll = 0, int rr =
24         MAX_V){
25         if (rr-ll==0) return;
26         if (rr-ll==1){
27             if (val.y(ll)<arr[idx].y(ll)){
28                 arr[idx] = val;
29             }
30             return;
31         }
32         int mid = (ll+rr)/2;
33         if (arr[idx].a > val.a) swap(arr[idx], val); // 原本
34             的線斜率要比較小
35         if (arr[idx].y(mid) < val.y(mid)){ // 交點在左邊
36             update(val, idx*2+1, ll, mid);
37         }else{ // 交點在右邊
38             swap(arr[idx], val); // 在左子樹中，新線比舊線還
39                 要好
40             update(val, idx*2+2, mid, rr);
41         }
42         return;
43     }
44
45     int query(int x, int idx = 0, int ll = 0, int rr = MAX_V)
46     {
47         if (rr-ll==0) return INF;
48         if (rr-ll==1){
49             return arr[idx].y(ll);
50         }
51         int mid = (ll+rr)/2;
52         if (x<mid){
53             return min(arr[idx].y(x), query(x, idx*2+1, ll,
54                 mid));
55         }else{

```

```

53         return min(arr[idx].y(x), query(x, idx*2+2, mid,
54             rr));
55     }
56 };

```

3.7 Segment Tree Li Chao Segment [cb9b37]

```

1 // 全部都是 0-based
2 // 宣告：LC_Segment_Tree st(n); // n = 值域大小 · x 的範圍為
3 // [0, n)
4 // 函式：
5 // st.update_segment({a, b}, ql, qr)：在 [ql, qr) 插入一條 y =
6 // ax+b 的線段
7 // st.query(x)：查詢所有線段在位置 x 的最小值
8
9 struct LC_Segment_Tree {
10     struct Line { // y = ax + b
11         int a = 0, b = INF;
12         int y(int x) const { return a * x + b; }
13     };
14     int N;
15     vector<Line> arr;
16     LC_Segment_Tree(int n) : N(n), arr(4 * n + 5) {}
17
18     void insert_line(Line val, int idx, int l, int r) {
19         if (arr[idx].a > val.a) swap(arr[idx], val);
20         if (arr[idx].a == val.a) {
21             arr[idx].b = min(arr[idx].b, val.b);
22             return;
23         }
24         if (val.y(l) >= arr[idx].y(l)) return;
25         if (val.y(r - 1) < arr[idx].y(r - 1)) {
26             arr[idx] = val;
27             return;
28         }
29         int mid = (l + r) / 2;
30         if (val.y(mid) < arr[idx].y(mid)) {
31             swap(arr[idx], val);
32             insert_line(val, 2 * idx + 2, mid, r);
33         }
34         else insert_line(val, 2 * idx + 1, l, mid);
35     }
36
37     void dfs(Line v, int ql, int qr, int idx, int l, int r) {
38         if (l >= r || r <= ql || l >= qr) return;
39         if (ql <= l && r <= qr) {
40             insert_line(v, idx, l, r);
41             return;
42         }
43         int mid = (l + r) / 2;
44         dfs(v, ql, qr, 2 * idx + 1, l, mid);
45         dfs(v, ql, qr, 2 * idx + 2, mid, r);
46     }
47
48     int q(int x, int idx, int l, int r) {
49         if (l >= r) return INF;
50         int res = arr[idx].y(x);
51         if (l + 1 == r) return res;
52         int mid = (l + r) / 2;
53         if (x < mid) return min(res, q(x, 2*idx+1, l, mid));
54         else return min(res, q(x, 2*idx+2, mid, r));
55     }

```

3.8 Segment Tree Persistent [3b5aa9]

```

1 // 全部都是 0-based
2 // Persistent_Segment_Tree st(n+q);
3 // st.build(v, 0);
4 // 函式：
5 // update_version(pos, val, ver)：對版本 ver 的 pos 位置改成
6 // val
7 // query_version(ql, qr, ver)：對版本 ver 查詢 [ql, qr) 的區
8 // 間和
9 // clone_version(ver)：複製版本 ver 到最新的版本
10
11 struct Persistent_Segment_Tree {
12     int node_cnt = 0;
13     struct Node {
14         int lc = -1;
15         int rc = -1;
16         int val = 0;
17     };
18     vector<Node> arr;
19     vector<int> version;
20
21     Persistent_Segment_Tree(int sz) {
22         arr.resize(32*sz);
23         version.push_back(node_cnt++);
24         return;
25     }
26
27     void pull(Node &c, Node a, Node b) {
28         c.val = a.val+b.val;
29         return;
30     }
31
32     void build(vector<int> &v, int idx, int ll = 0, int rr =
33         n) {
34         auto &now = arr[idx];
35
36         if (rr-ll==1) {
37             now.val = v[ll];
38             return;
39         }
40
41         int mid = (ll+rr)/2;
42         now.lc = node_cnt++;
43         now.rc = node_cnt++;
44         build(v, now.lc, ll, mid);
45         build(v, now.rc, mid, rr);
46         pull(now, arr[now.lc], arr[now.rc]);
47         return;
48     }
49
50     void update(int pos, int val, int idx, int ll = 0, int rr
51         = n) {
52         auto &now = arr[idx];
53
54         if (rr-ll==1) {
55             now.val = val;
56             return;
57         }
58
59         int mid = (ll+rr)/2;
60         now.lc = node_cnt++;
61         now.rc = node_cnt++;
62         build(v, now.lc, ll, mid);
63         build(v, now.rc, mid, rr);
64         pull(now, arr[now.lc], arr[now.rc]);
65         return;
66     }
67
68     void query(int ql, int qr, int ver) {
69         int mid = (ll+rr)/2;
70         if (pos < mid) {
71             arr[node_cnt] = arr[now.lc];
72             now.lc = node_cnt++;
73             update(pos, val, now.lc, ll, mid);
74         }
75         else {
76             arr[node_cnt] = arr[now.rc];
77             now.rc = node_cnt++;
78             update(pos, val, now.rc, mid, rr);
79         }
80         pull(now, arr[now.lc], arr[now.rc]);
81         return;
82     }
83
84     void update_version(int pos, int val, int ver) {
85         update(pos, val, version[ver]);
86     }
87
88     Node query(int ql, int qr, int idx, int ll = 0, int rr =
89         n) {
90         auto &now = arr[idx];
91
92         if (ql <= ll && rr <= qr) return now;
93         if (rr <= ql || qr <= ll) return Node();
94
95         int mid = (ll+rr)/2;
96         Node ret;
97         pull(ret, query(ql, qr, now.lc, ll, mid), query(ql,
98             qr, now.rc, mid, rr));
99         return ret;
100     }
101
102     Node query_version(int ql, int qr, int ver) {
103         return query(ql, qr, version[ver]);
104     }
105
106     void clone_version(int ver) {
107         version.push_back(node_cnt);
108         arr[node_cnt] = arr[version[ver]];
109         node_cnt++;
110     }
111 };

```

```

54     int mid = (ll+rr)/2;
55     if (pos < mid) {
56         arr[node_cnt] = arr[now.lc];
57         now.lc = node_cnt++;
58         update(pos, val, now.lc, ll, mid);
59     }
60     else {
61         arr[node_cnt] = arr[now.rc];
62         now.rc = node_cnt++;
63         update(pos, val, now.rc, mid, rr);
64     }
65     pull(now, arr[now.lc], arr[now.rc]);
66     return;
67 }
68
69 void update_version(int pos, int val, int ver) {
70     update(pos, val, version[ver]);
71 }
72
73 Node query(int ql, int qr, int idx, int ll = 0, int rr =
74     n) {
75     auto &now = arr[idx];
76
77     if (ql <= ll && rr <= qr) return now;
78     if (rr <= ql || qr <= ll) return Node();
79
80     int mid = (ll+rr)/2;
81     Node ret;
82     pull(ret, query(ql, qr, now.lc, ll, mid), query(ql,
83         qr, now.rc, mid, rr));
84     return ret;
85 }
86
87 Node query_version(int ql, int qr, int ver) {
88     return query(ql, qr, version[ver]);
89 }
90
91 void clone_version(int ver) {
92     version.push_back(node_cnt);
93     arr[node_cnt] = arr[version[ver]];
94     node_cnt++;
95 }
96 };

```

3.9 Segment Tree Zkw [69ce93]

```

1 struct SegmentTreeZkw { // 1-based index
2     using T = array<int, 2>;
3     using U = int;
4     int n, m;
5     vector<T> seg;
6     vector<U> lazy;
7     static constexpr T empty_node = {0, 0};
8     static constexpr U empty_lazy = 1;
9
10     SegmentTreeZkw(int sz) : n(sz), m(sz - 1) {
11         seg.assign(2 * n, empty_node);
12         lazy.assign(2 * n, empty_lazy);
13     }
14     void build(T* a) {
15         for (int i = 1; i <= n; ++i) {
16             seg[m + i] = a[i];
17         }
18     }

```

```

18     for (int i = m; i >= 1; --i) {
19         seg[i] = merge(seg[i << 1], seg[i << 1 | 1]);
20     }
21 }
22 void range_update(int l, int r, U val) {
23     l += m; r += m;
24     int l0 = l, r0 = r;
25     push(l0), push(r0);
26     while (l <= r) {
27         if (l & 1) modify(l++, val);
28         if (!(r & 1)) modify(r--, val);
29         l >>= 1; r >>= 1;
30     }
31     push(l0), push(r0);
32     pull(l0), pull(r0);
33 }
34 void modify(int i, U val) {
35     seg[i][1] = seg[i][1] * val % mod;
36     lazy[i] = lazy[i] * val % mod;
37 }
38 void push(int i) {
39     for (int h = __lg(i); h >= 1; h--) {
40         int idx = i >> h;
41         if (lazy[idx] == empty_lazy) continue;
42         modify(idx << 1, lazy[idx]);
43         modify(idx << 1 | 1, lazy[idx]);
44         lazy[idx] = empty_lazy;
45     }
46 }
47 void pull(int i) {
48     while (i >>= 1) {
49         seg[i] = merge(seg[i << 1], seg[i << 1 | 1]);
50     }
51 }
52 T merge(T x, T y) {
53     return {x[0] + y[0], (x[1] + hmul_pow[x[0]] * y[1]) %
54         mod};
55 }
56 T query(int l, int r) { // [l, r]
57     T resl = empty_node, resr = empty_node;
58     l += m; r += m;
59     push(l), push(r);
60     while (l <= r) {
61         if (l & 1) resl = merge(resl, seg[l++]);
62         if (!(r & 1)) resr = merge(seg[r--], resr);
63         l >>= 1; r >>= 1;
64     }
65     return merge(resl, resr);
66 };

```

3.10 Sparse Table [31f22a]

```

1 struct SparseTable{
2     vector<vector<int>> st;
3     void build(vector<int> v){
4         int h = __lg(v.size());
5         st.resize(h+1);
6         st[0] = v;
7
8         for (int i=1; i<=h; i++){
9             int gap = (1<<(i-1));
10            for (int j=0; j+gap<st[i-1].size(); j++){
11                st[i].push_back(min(st[i-1][j], st[i-1][j+gap
12                ]));

```

```

12     }
13 }
14 }
15
16 // 回傳 [ll, rr] 的最小值
17 int query(int ll, int rr){
18     int h = __lg(rr-ll);
19     return min(st[h][ll], st[h][rr-(1<<h)]);
20 }
21 };

```

3.11 Treap2 [3b0cca]

```

1 // 1-based · 請注意 MAX_N 是否足夠大
2 int root = 0;
3 int lc[MAX_N], rc[MAX_N];
4 int pri[MAX_N], val[MAX_N];
5 int sz[MAX_N], tag[MAX_N], fa[MAX_N], total[MAX_N];
6 // tag 為不包含自己 (僅要給子樹) 的資訊
7 int nodeCnt = 0;
8 int& new_node(int v){
9     nodeCnt++;
10    val[nodeCnt] = v;
11    total[nodeCnt] = v;
12    sz[nodeCnt] = 1;
13    pri[nodeCnt] = rand();
14    return nodeCnt;
15 }
16
17 void apply(int x, int V){
18     val[x] += V;
19     tag[x] += V;
20     total[x] += V*sz[x];
21 }
22
23 void push(int x){
24     if (tag[x]){
25         if (lc[x]) apply(lc[x], tag[x]);
26         if (rc[x]) apply(rc[x], tag[x]);
27     }
28     tag[x] = 0;
29 }
30
31 int pull(int x){
32     if (x){
33         fa[x] = 0;
34         sz[x] = 1+sz[lc[x]]+sz[rc[x]];
35         total[x] = val[x]+total[lc[x]]+total[rc[x]];
36         if (lc[x]) fa[lc[x]] = x;
37         if (rc[x]) fa[rc[x]] = x;
38     }
39     return x;
40 }
41
42 int merge(int a, int b){
43     if (!a or !b) return a|b;
44     push(a), push(b);
45
46     if (pri[a]>pri[b]){
47         rc[a] = merge(rc[a], b);
48         return pull(a);
49     }else{
50         lc[b] = merge(a, lc[b]);
51         return pull(b);
52 }

```

```

53
54 // [1, k] [k+1, n]
55 void split(int x, int k, int &a, int &b) {
56     if (!x) return a = b = 0, void();
57     push(x);
58     if (sz[lc[x]] >= k) {
59         split(lc[x], k, a, lc[x]);
60         b = x;
61         pull(a); pull(b);
62     }else{
63         split(rc[x], k - sz[lc[x]] - 1, rc[x], b);
64         a = x;
65         pull(a); pull(b);
66     }
67 }
68
69 // functions
70 // 回傳 x 在 Treap 中的位置
71 int get_pos(int x){
72     vector<int> sta;
73     while (fa[x]){
74         sta.push_back(x);
75         x = fa[x];
76     }
77     while (sta.size()){
78         push(x);
79         x = sta.back();
80         sta.pop_back();
81     }
82     push(x);
83
84     int res = sz[x] - sz[rc[x]];
85     while (fa[x]){
86         if (rc[fa[x]]==x){
87             res += sz[fa[x]]-sz[x];
88         }
89         x = fa[x];
90     }
91     return res;
92 }
93
94 // 1-based <前 [1, l-1] 個元素, [l, r] 個元素, [r+1, n] 個元
95 素>
96 array<int, 3> cut(int x, int l, int r){
97     array<int, 3> ret;
98     split(x, r, ret[1], ret[2]);
99     split(ret[1], l-1, ret[0], ret[1]);
100    return ret;
101 }
102
103 void print(int x){
104     push(x);
105     if (lc[x]) print(lc[x]);
106     cerr << val[x] << " ";
107     if (rc[x]) print(rc[x]);

```

3.12 Trie [b6475c]

```

1 struct Trie{
2     struct Data{
3         int nxt[2]={0, 0};
4     };
5
6     int sz=0;

```

```

7   vector<Data> arr;
8
9   void init(int n){
10       arr.resize(n);
11   }
12
13   void insert(int n){
14       int now=0;
15       for (int i=N ; i>=0 ; i--){
16           int v=(n>>i)&1;
17           if (!arr[now].nxt[v]){
18               arr[now].nxt[v]=++sz;
19           }
20           now=arr[now].nxt[v];
21       }
22   }
23
24   int query(int n){
25       int now=0, ret=0;
26       for (int i=N ; i>=0 ; i--){
27           int v=(n>>i)&1;
28           if (arr[now].nxt[1-v]){
29               ret+=(1<<i);
30               now=arr[now].nxt[1-v];
31           }else if (arr[now].nxt[v]){
32               now=arr[now].nxt[v];
33           }else{
34               return ret;
35           }
36       }
37       return ret;
38   }
39 } tr;

```

4 Dynamic-Programming

4.1 Digit DP [133f00]

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  long long l, r;
5  long long dp[20][10][2][2]; // dp[pos][pre][limit] = 後 pos
6  // 位, pos 前一位是 pre, (是/否) 有上界, (是/否) 有前綴零
7  // 的答案數量
8
9  long long memorize_search(string &s, int pos, int pre, bool
10 limit, bool lead){
11
12     // 已經被找過了, 直接回傳值
13     if (dp[pos][pre][limit][lead]!=-1) return dp[pos][pre][
14 limit][lead];
15
16     // 已經搜尋完畢, 紀錄答案並回傳
17     if (pos==(int)s.size()){
18         return dp[pos][pre][limit][lead] = 1;
19     }
20
21     // 枚舉目前的位數數字是多少
22     long long ans = 0;
23     for (int now=0 ; now<=(limit ? s[pos]-'0' : 9) ; now++){
24         if (now==pre){
25             // 1~9 絕對不能連續出現

```

```

23         if (pre!=0) continue;
24
25         // 如果已經不在前綴零的範圍內, 0 不能連續出現
26         if (lead==false) continue;
27     }
28
29     ans += memorize_search(s, pos+1, now, limit&(now==(s[
30 pos]-'0'))), lead&(now==0));
31 }
32
33 // 已經搜尋完畢, 紀錄答案並回傳
34 return dp[pos][pre][limit][lead] = ans;
35 }
36
37 // 回傳 [0, n] 有多少數字符合條件
38 long long find_answer(long long n){
39     memset(dp, -1, sizeof(dp));
40     string tmp = to_string(n);
41
42     return memorize_search(tmp, 0, 0, true, true);
43 }
44
45 int main(){
46     // input
47     cin >> l >> r;
48
49     // output - 計算 [l, r] 有多少數字任意兩個位數都不相同
50     cout << find_answer(r)-find_answer(l-1) << "\n";
51
52     return 0;
53 }

```

4.2 Integer Partition

$dp[i][x]$ = 要將整數 x 拆成 i 堆的「組合數」

$dp[i+1][x+1] + = dp[i][x]$ (創造新的一堆)
 $dp[i][x+i] + = dp[i][x]$ (把每一堆都增加 1)

4.3 Knapsack On Tree [df69b1]

```

1  // 需要重構、需要增加註解
2  #include <bits/stdc++.h>
3  #define F first
4  #define S second
5  #define all(x) begin(x), end(x)
6  using namespace std;
7
8  #define chmax(a, b) (a) = (a) < (b) ? (b) : (a)
9  #define chmin(a, b) (a) = (a) < (b) ? (a) : (b)
10
11 #define ll long long
12
13 #define FOR(i, a, b) for (int i = a; i <= b; i++)
14
15 int N, W, cur;
16 vector<int> w, v, sz;
17 vector<vector<int>> adj, dp;
18
19 void dfs(int x) {
20     sz[x] = 1;
21     for (int i : adj[x]) dfs(i), sz[x] += sz[i];
22     cur++;
23     // choose x

```

```

24     for (int i=w[x] ; i<=W ; i++){
25         dp[cur][i] = dp[cur - 1][i - w[x]] + v[x];
26     }
27     // not choose x
28     for (int i=0 ; i<=W ; i++){
29         chmax(dp[cur][i], dp[cur - sz[x]][i]);
30     }
31 }
32
33 signed main() {
34     cin >> N >> W;
35     adj.resize(N + 1);
36     w.assign(N + 1, 0);
37     v.assign(N + 1, 0);
38     sz.assign(N + 1, 0);
39     dp.assign(N + 2, vector<int>(W + 1, 0));
40     for (int i=1 ; i<=N ; i++){
41         int p; cin >> p;
42         adj[p].push_back(i);
43     }
44
45     for (int i=1 ; i<=N ; i++) cin >> w[i];
46     for (int i=1 ; i<=N ; i++) cin >> v[i];
47     dfs(0);
48     cout << dp[N + 1][W] << "\n";
49 }

```

4.4 SOS DP [8dfa8b]

```

1  // 總時間複雜度為  $O(n \cdot 2^n)$ 
2  // 計算  $dp[i] = i$  所有 bit mask 子集的和
3  for (int i=0 ; i<n ; i++){
4      for (int mask=0 ; mask<(1<<n) ; mask++){
5          if ((mask>>i)&1){
6              dp[mask] += dp[mask^(1<<i)];
7          }
8      }
9  }

```

5 Geometry

5.1 misc [3ef351]

```

1  using ld = double;
2
3  // 判斷數值正負: {1:正數,0:零,-1:負數}
4  int sign(long long x) {return (x >= 0) ? ((bool)x) : -1; }
5  int sign(ld x) {return (abs(x) < 1e-9) ? 0 : (x>0 ? 1 : -1); }

```

5.2 convex hull [bbcaaf]

```

1  int sign(long long x) {return (x >= 0) ? ((bool)x) : -1; }
2
3  template<typename T>
4  struct point {
5      T x, y;
6      point() {}
7      point(const T &x0, const T &y0) : x(x0), y(y0) {}
8
9      point operator-(point b) {return {x-b.x, y-b.y}; }
10     bool operator==(point b) {return x==b.x && y==b.y; }
11     T operator^(point b) {return x * b.y - y * b.x; }
12     friend int ori(point a, point b, point c) {
13         return sign((b-a)^(c-a));

```

```

14 }
15 };
16
17 template<typename T>
18 struct polygon {
19     vector<point<T>> v;
20     void make_convex_hull(int simple) {
21         auto cmp = [&](point<T> &p, point<T> &q) {
22             return (p.x == q.x) ? (p.y < q.y) : (p.x < q.x);
23         };
24         simple = (bool)simple;
25         sort(v.begin(), v.end(), cmp);
26         v.resize(unique(v.begin(), v.end()) - v.begin());
27         if (v.size() <= 1) return;
28         vector<point<T>> hull;
29         for (int t = 0; t < 2; ++t){
30             int sz = hull.size();
31             for (auto &i:v) {
32                 while (hull.size() >= sz+2 && ori(hull[hull.
33                     size()-2], hull.back(), i) < simple) {
34                     hull.pop_back();
35                 }
36                 hull.push_back(i);
37             }
38             hull.pop_back();
39             reverse(v.begin(), v.end());
40         }
41         swap(hull, v);
42     }
43 };

```

5.3 SVG writer [5bf313]

```

1 class SVG {
2     void p(string_view s) { o << s; }
3     void p(string_view s, auto v, auto... vs) {
4         auto i = s.find('$');
5         o << s.substr(0, i) << v, p(s.substr(i + 1), vs...);
6     }
7     ofstream o;
8     string c = "red";
9 public:
10     SVG(auto f, auto x1, auto y1, auto x2, auto y2) : o(f) {
11         p("<svg xmlns='http://www.w3.org/2000/svg' "
12           "viewBox='$ $ $ $'\n"
13           "<style>*<stroke-width:0.5%;</style>\n",
14           x1, -y2, x2 - x1, y2 - y1);
15     } // cc67e3
16     ~SVG() { p("</svg>\n"); }
17     void color(string nc) { c = nc; }
18     void line(auto x1, auto y1, auto x2, auto y2) {
19         p("<line x1='$' y1='$' x2='$' y2='$' stroke='$' />\n",
20           x1, -y1, x2, -y2, c);
21     } // a08d95
22     void circle(auto x, auto y, auto r) {
23         p("<circle cx='$' cy='$' r='$' stroke='$' "
24           "fill='$' />\n", x, -y, r, c, "#none");
25     } // 73c400
26     void text(auto x, auto y, string s, int w = 12) {
27         p("<text x='$' y='$' font-size='$px'>${</text>\n",
28           x, -y, w, s);
29     } // a23944
30 };
31 // declare : SVG svg("test.svg", -99, -99, 100, 100);

```

5.4 struct point [f21f07]

```

1 template<typename T>
2 struct point {
3     T x, y;
4     point() {}
5     point(const T &x0, const T &y0) : x(x0), y(y0) {}
6     explicit operator point<ld>() {return point<ld>(x, y); }
7 };

```

5.5 point / basic operation [c415da]

```

1 point operator+(point b) {return {x+b.x, y+b.y}; }
2 point operator-(point b) {return {x-b.x, y-b.y}; }
3 point operator*(T b) {return {x*b, y*b}; }
4 point operator/(T b) {return {x/b, y/b}; }
5 bool operator==(point b) {return x==b.x && y==b.y; }
6
7 T operator*(point b) {return x * b.x + y * b.y; }
8 T operator^(point b) {return x * b.y - y * b.x; }

```

5.6 point / operators [fc43e4]

```

1 // 逆時針極角排序
2 bool side() const{return (y == 0) ? (x > 0) : (y < 0); }
3 bool operator<(const point &b) const {
4     return side() == b.side() ?
5         (x*b.y > b.x*y) : side() < b.side();
6 }
7 friend ostream& operator<<(ostream& os, point p) {
8     return os << "(" << p.x << ", " << p.y << ")";
9 }
10 // 判斷 ab 到 ac 的方向 : {1:逆時鐘,0:重疊,-1:順時鐘}
11 friend int ori(point a, point b, point c) {
12     return sign((b-a)^(c-a));
13 }
14 friend bool btw(point a, point b, point c) {
15     return ori(a, b, c) == 0 && sign((a-c)*(b-c)) <= 0;
16 }

```

5.7 point / banana [09fd7c]

```

1 // 判斷線段 ab, cd 是否相交
2 friend bool banana(point a, point b, point c, point d) {
3     if (btw(a, b, c) || btw(a, b, d)
4         || btw(c, d, a) || btw(c, d, b)) return true;
5     int u = ori(a, b, c) * ori(a, b, d);
6     int v = ori(c, d, a) * ori(c, d, b);
7     return u < 0 && v < 0;
8 }

```

5.8 point / rayHitSeg [db541a]

```

1 // 判斷 "射線 ab" 與 "線段 cd" 是否相交
2 friend bool rayHitSeg(point a, point b, point c, point d) {
3     if (a == b) return btw(c, d, a); // Special case
4     if (((a - b) ^ (c - d)) == 0) {
5         return btw(a, c, b) || btw(a, d, b) || banana(a, b, c
6             , d);
7     }
8     point u = b - a, v = d - c, s = c - a;
9     return sign(s ^ v) * sign(u ^ v) >= 0 && sign(s ^ u)
10        * sign(u ^ v) >= 0 && abs(s ^ u) <= abs(u ^ v);

```

5.9 point / else [f46547]

```

1 // 旋轉 Arg(b) 的角度 (小心溢位)
2 point rotate(point b){return {x*b.x-y*b.y, x*b.y+y*b.x};}
3 // 回傳極座標角度·值域: [-π, +π]
4 friend ld Arg(point b) {
5     return (b.x != 0 || b.y != 0) ? atan2(b.y, b.x) : 0;
6 }
7 friend T abs2(point b) {return b * b; }

```

5.10 struct line [210dc8]

```

1 template<typename T>
2 struct line {
3     point<T> p1, p2;
4     // ax + by + c = 0
5     T a, b, c; // |a|, |b| ≤ 2C, |c| ≤ 8C²
6     line() {}
7     line(const point<T> &x, const point<T> &y) : p1(x), p2(y){
8         build();
9     }
10    void build() {
11        a = p1.y - p2.y;
12        b = p2.x - p1.x;
13        c = (-a*p1.x)-b*p1.y;
14    }
15 };

```

5.11 line / else [09bcc9]

```

1 // 判斷點和有向直線的關係 : {1:左邊,0:在線上,-1:右邊}
2 int ori(point<T> &p) {
3     return sign((p2-p1) ^ (p-p1));
4 }
5 // 判斷直線斜率是否相同
6 bool parallel(line &l) {
7     return ((p1-p2) ^ (l.p1-l.p2)) == 0;
8 }
9 // 兩直線交點
10 point<ld> line_intersection(line &l) {
11     using P = point<ld>;
12     point<T> u = p2-p1, v = l.p2-l.p1, s = l.p1-p1;
13     return P(p1) + P(u) * ((ld(s^v)) / (u^v));
14 }

```

5.12 struct polygon [171a85]

```

1 template<typename T>
2 struct polygon {
3     vector<point<T>> v;
4     polygon() {}
5     polygon(const vector<point<T>> &u) : v(u) {}
6 };

```

5.13 polygon / make convex hull [2bb3ef]

```

1 // simple 為 true 的時候會回傳任意三點不共線的凸包
2 void make_convex_hull(int simple) {
3     auto cmp = [&](point<T> &p, point<T> &q) {
4         return (p.x == q.x) ? (p.y < q.y) : (p.x < q.x);
5     };
6     simple = (bool)simple;
7     sort(v.begin(), v.end(), cmp);
8     v.resize(unique(v.begin(), v.end()) - v.begin());
9     if (v.size() <= 1) return;
10    vector<point<T>> hull;
11    for (int t = 0; t < 2; ++t){
12        int sz = hull.size();
13        for (auto &i:v) {
14            while (hull.size() >= sz+2 && ori(hull[hull.size()-2], hull.back(), i) < simple) {
15                hull.pop_back();
16            }
17            hull.push_back(i);
18        }
19        hull.pop_back();
20        reverse(v.begin(), v.end());
21    }
22    swap(hull, v);
23 }

```

5.14 polygon / in polygon [f11340]

```

1 // 可以在有 n 個點的簡單多邊形內，用 O(n) 判斷一個點：
2 // {1 : 在多邊形內, 0 : 在多邊形上, -1 : 在多邊形外}
3 int in_polygon(point<T> a){
4     const T MAX_POS = 1e9 + 5; // [記得修改] 座標的最大值
5     point<T> pre = v.back(), b(MAX_POS, a.y + 1);
6     int cnt = 0;
7
8     for (auto &i:v) {
9         if (btw(pre, i, a)) return 0;
10        if (banana(a, b, pre, i)) cnt++;
11        pre = i;
12    }
13
14    return cnt%2 ? 1 : -1;
15 }

```

5.15 polygon / in convex [e64f1e]

```

1 /// 警告：以下所有凸包專用的函式都只接受逆時針排序且任三點不
2 /// 共線的凸包 ///
3 // 可以在有 n 個點的凸包內，用 O(Log n) 判斷一個點：
4 // {1 : 在凸包內, 0 : 在凸包邊上, -1 : 在凸包外}
5 int in_convex(point<T> p) {
6     int n = v.size();
7     int a = ori(v[0], v[1], p), b = ori(v[0], v[n-1], p);
8     if (a < 0 || b > 0) return -1;
9     if (btw(v[0], v[1], p)) return 0;
10    if (btw(v[0], v[n-1], p)) return 0;
11    int l = 1, r = n - 1, mid;
12    while (l + 1 < r) {
13        mid = (l + r) >> 1;
14        if (ori(v[0], v[mid], p) >= 0) l = mid;
15        else r = mid;
16    }
17    int k = ori(v[l], v[r], p);

```

```

17     if (k <= 0) return k;
18     return 1;
19 }

```

5.16 polygon / cycle search [fe2f51]

```

1 // 凸包專用的環狀二分搜，回傳 0-based index
2 int cycle_search(auto &f) {
3     int n = v.size(), l = 0, r = n;
4     if (n == 1) return 0;
5     bool rv = f(1, 0);
6     while (r - l > 1) {
7         int m = (l + r) / 2;
8         if (f(0, m) ? rv : f(m, (m + 1) % n)) r = m;
9         else l = m;
10    }
11    return f(1, r % n) ? l : r % n;
12 }

```

5.17 polygon / line cut convex [b6a4c8]

```

1 // 可以在有 n 個點的凸包內，用 O(Log n) 判斷一條直線：
2 // {1 : 穿過凸包, 0 : 剛好切過凸包, -1 : 沒碰到凸包}
3 int line_cut_convex(line<T> L) {
4     L.build();
5     point<T> p(L.a, L.b);
6     auto gt = [&](int neg) {
7         auto f = [&](int x, int y) {
8             return sign((v[x] - v[y]) * p) == neg;
9         };
10        return -(v[cycle_search(f)] * p);
11    };
12    T x = gt(1), y = gt(-1);
13    if (L.c < x || y < L.c) return -1;
14    return not (L.c == x || L.c == y);
15 }

```

5.18 polygon / convex line intersect [47e699]

```

1 // return edge endpoint index
2 // but if (first == second) => intersect at a vertex
3 vector<pair<int,int>> convex_line_intersect(line<T> L) {
4     auto dis = [&](int p){
5         return (L.p2 - L.p1) ^ (v[p] - L.p1);
6     };
7     auto gao = [&](int s) {
8         auto f = [&](int i, int j) {
9             return sign(dis(i) - dis(j)) == s;
10        };
11        return cycle_search(f);
12    };
13    int x = gao(1), y = gao(-1), n = v.size();
14    if (sign(dis(x)) < 0 || sign(dis(y)) > 0) return {};
15    if (sign(dis(x)) == 0 || sign(dis(y)) == 0) {
16        int j = ((sign(dis(x)) == 0 ? x : y) + n - 1) % n;
17        vector<pair<int,int>> ret;
18        for (int i = 0; i < 3; ++i, j = (j + 1) % n)
19            if (sign(dis(j)) == 0) ret.emplace_back(j, j);
20        return ret;
21    }
22    auto g = [&](int l, int r, int s) -> pair<int,int> {
23        while ((l + 1) % n != r) {
24            int m = ((l + r + (l < r ? 0 : n)) / 2) % n;
25            (sign(dis(m)) == s ? l : r) = m;

```

```

26        }
27        return {sign(dis(r)) ? l : r, r};
28    };
29    return {g(x, y, 1), g(y, x, -1)};
30 }

```

5.19 polygon / segment pass convex interior [7a3f22]

```

1 // 可以在有 n 個點的凸包內，用 O(Log n) 判斷一個線段：
2 // {1 : 線段上存在某一點位於凸包內部（邊上不算），
3 // 0 : 線段上存在某一點碰到凸包的邊但線段上任一點均不在凸包
4 // 內部，
5 // -1 : 線段完全在凸包外面}
6 int segment_pass_convex_interior(line<T> L) {
7     if (in_convex(L.p1) == 1 || in_convex(L.p2) == 1) return
8         1;
9     L.build();
10    auto res = convex_line_intersect(L);
11    if (res.empty()) return -1;
12    bool cntp = 0, cntn = 0;
13    for (auto &[i, j] : res) if (banana(v[i], v[j], L.p1, L.p2)) {
14        int now = L.ori(v[(i + 1) % v.size()]);
15        cntp |= (now == 1);
16        cntn |= (now == -1);
17    }
18    return (-1 + cntp + cntn);

```

5.20 polygon / convex tangent point [a6f66b]

```

1 // 回傳點過凸包的兩條切線的切點的 0-based index (不保證兩條
2 // 切線的順逆時針關係)
3 pair<int,int> convex_tangent_point(point<T> p) {
4     int n = v.size(), z = -1, edg = -1;
5     auto gt = [&](int neg) {
6         auto check = [&](int x) {
7             if (v[x] == p) z = x;
8             if (btw(v[x], v[(x + 1) % n], p)) edg = x;
9             if (btw(v[(x + n - 1) % n], v[x], p)) edg = (x +
10                 n - 1) % n;
11         };
12         auto f = [&](int x, int y) {
13             check(x); check(y);
14             return ori(p, v[x], v[y]) == neg;
15         };
16         return cycle_search(f);
17     };
18    int x = gt(1), y = gt(-1);
19    if (z != -1) {
20        return {(z + n - 1) % n, (z + 1) % n};
21    }
22    else if (edg != -1) {
23        return {edg, (edg + 1) % n};
24    }
25    else {
26        return {x, y};
27    }

```

5.21 polygon / halfplane intersection [102d48]

```

1 friend int halfplane_intersection(vector<line<T>> &s, polygon
  <T> &P) {
2     auto angle_cmp = [&](line<T> &A, line<T> &B) {
3         point<T> a = A.p2-A.p1, b = B.p2-B.p1;
4         return (a < b);
5     };
6     sort(s.begin(), s.end(), angle_cmp); // 線段左側為該線段
      半平面
7     int L, R, n = s.size();
8     vector<point<T>> px(n);
9     vector<line<T>> q(n);
10    q[L = R = 0] = s[0];
11    for(int i = 1; i < n; ++i) {
12        while(L < R && s[i].ori(px[R-1]) <= 0) --R;
13        while(L < R && s[i].ori(px[L]) <= 0) ++L;
14        q[++R] = s[i];
15        if(q[R].parallel(q[R-1])) {
16            --R;
17            if(q[R].ori(s[i].p1) > 0) q[R] = s[i];
18        }
19        if(L < R) px[R-1] = q[R-1].line_intersection(q[R]);
20    }
21    while(L < R && q[L].ori(px[R-1]) <= 0) --R;
22    P.v.clear();
23    if(R - L <= 1) return 0;
24    px[R] = q[R].line_intersection(q[L]);
25    for(int i = L; i <= R; ++i) P.v.push_back(px[i]);
26    return R - L + 1;
27 }

```

5.22 polygon / minkowski sum [aba519]

```

1 void minkowski(vector<point<T>> a, vector<point<T>> b) {
2     // a, b: convex polygon's inside vector, CCW order
3     auto cmp = [&](auto &a, auto &b) {
4         return tie(a.y, a.x) < tie(b.y, b.x);
5     };
6     auto reorder = [&](auto &u) {
7         auto it = min_element(u.begin(), u.end(), cmp);
8         rotate(u.begin(), it, u.end());
9     };
10    reorder(a); reorder(b);
11    a.push_back(a[0]); a.push_back(a[1]);
12    b.push_back(b[0]); b.push_back(b[1]);
13    v.clear();
14    for (int i = 0, j = 0; i+2<a.size()||j+2<b.size();) {
15        v.push_back(a[i] + b[j]);
16        auto val = (a[i + 1] - a[i]) ^ (b[j + 1] - b[j]);
17        if (val >= 0) i++;
18        if (val <= 0) j++;
19    }
20 }

```

5.23 struct Cir [87198d]

```

1 struct Cir {
2     point<ld> o; ld r;
3     friend ostream& operator<<(ostream& os, Cir c) {
4         return os << "(" << "x" << "+" << [c.o.x >= 0] << abs(c.o.x)
          << ")^2 + (y" << "+" << [c.o.y >= 0] << abs(c.o.y)
          << ")^2 = " << c.r * c.r;
5     }

```

```

6     bool covers(Cir b) {
7         return sqrt((ld)abs2(o - b.o)) + b.r <= r;
8     }
9 };

```

5.24 Cir / Cir intersect [330a1c]

```

1 vector<point<ld>> Cir_intersect(Cir c) {
2     ld d2 = abs2(o - c.o), d = sqrt(d2);
3     if (d < max(r, c.r) - min(r, c.r) || d > r + c.r) return
      {};
4     auto sqdf = [&](ld x, ld y) { return x*x - y*y; };
5     point<ld> u = (o + c.o) / 2 + (o - c.o) * (sqdf(c.r, r) /
      (2 * d2));
6     ld A = sqrt(sqdf(r + d, c.r) * sqdf(c.r, d - r));
7     point<ld> v = (c.o - o).rotate({0,1}) * A / (2 * d2);
8     if (sign(v.x) == 0 && sign(v.y) == 0) return {u};
9     return {u - v, u + v};
10 }

```

5.25 Cir / point tangent [0067e6]

```

1 auto point_tangent(point<ld> p) {
2     vector<point<ld>> res;
3     ld d_sq = abs2(p - o);
4     if (sign(d_sq - r * r) <= 0) {
5         res.pb(p + (p - o).rotate({0, 1}));
6     } else if (d_sq > r * r) {
7         ld s = d_sq - r * r;
8         point<ld> v = p + (o - p) * s / d_sq;
9         point<ld> u = (o - p).rotate({0, 1}) * sqrt(s) * r /
          d_sq;
10        res.pb(v + u);
11        res.pb(v - u);
12    }
13    return res;
14 }

```

5.26 Cir / circle tangent [a3c7fb]

```

1 // returns nothing when two circles are the same
2 auto circle_tangent(Cir c2, int sign1) {
3     // sign1 = 1 for outer tang, -1 for inter tang
4     using ptld = point<ld>;
5     vector<pair<ptld, ptld>> res;
6     ld d_sq = abs2(o - c2.o);
7     if (sign(d_sq) == 0) return res;
8     ld d = sqrt(d_sq);
9     ptld v = (c2.o - o) / d;
10    ld c = (r - sign1 * c2.r) / d;
11    if (c * c > 1) return res;
12    ld h = sqrt(max((ld)0.0, 1.0 - c * c));
13    for (int sign2 = 1; sign2 >= -1; sign2 -= 2) {
14        ptld n(v.x * c - sign2 * h * v.y, v.y * c + sign2 * h
          * v.x);
15        ptld p1 = o + n * r;
16        ptld p2 = c2.o + n * (c2.r * sign1);
17        if (sign(p1.x - p2.x) == 0 && sign(p1.y - p2.y) == 0)
18            p2 = p1 + (c2.o - o).rotate({0, 1});
19        res.push_back({p1, p2});
20    }
21    if (sign1 == -1 && sign(r + c2.r - d) == 0) res.pop_back
      ();

```

```

22    if (sign1 == 1 && sign(abs(r - c2.r) - d) == 0) res.
      pop_back();
23    return res;
24 }

```

5.27 Cir / minimum enclosing circle [3e1440]

```

1 point<ld> circenter(point<ld> p0, point<ld> p1, point<ld> p2)
  {
2     // radius = abs(center)
3     p1 = p1 - p0, p2 = p2 - p0;
4     ld x1 = p1.x, y1 = p1.y, x2 = p2.x, y2 = p2.y;
5     ld m = 2. * (x1 * y2 - y1 * x2);
6     point<ld> center(0, 0);
7     center.x = (x1 * x1 * y2 - x2 * x2 * y1 + y1 * y2 * (y1 -
      y2)) / m;
8     center.y = (x1 * x2 * (x2 - x1) - y1 * y1 * x2 + x1 * y2
      * y2) / m;
9     return center + p0;
10 }
11 void min_enclosing(vector<point<ld>> p) {
12     shuffle(p.begin(), p.end(), rngf);
13     r = 0.0;
14     point<ld> cent = p[0];
15     for (int i = 1; i < p.size(); ++i) {
16         if (abs2(cent - p[i]) <= r) continue;
17         cent = p[i], r = 0.0;
18         for (int j = 0; j < i; ++j) {
19             if (abs2(cent - p[j]) <= r) continue;
20             cent = (p[i] + p[j]) / 2, r = abs2(p[j] - cent);
21             for (int k = 0; k < j; ++k) {
22                 if (abs2(cent - p[k]) <= r) continue;
23                 cent = circenter(p[i], p[j], p[k]);
24                 r = abs2(p[k] - cent);
25             }
26         }
27     }
28     o = cent;
29     r = sqrt(r);
30 }

```

5.28 Geometry Struct 3D [4a50c9]

```

1 using ld = long double;
2
3 // 判斷數值正負：{1:正數,0:零,-1:負數}
4 int sign(long long x) {return (x >= 0) ? ((bool)x) : -1; }
5 int sign(ld x) {return (abs(x) < 1e-9) ? 0 : (x>0 ? 1 : -1);}
6
7 template<typename T>
8 struct pt3 {
9     T x, y, z;
10    pt3(){}
11    pt3(const T &x, const T &y, const T &z):x(x),y(y),z(z){}
12    explicit operator pt3<ld>() {return pt3<ld>(x, y, z); }
13
14    pt3 operator+(pt3 b) {return {x+b.x, y+b.y, z+b.z}; }
15    pt3 operator-(pt3 b) {return {x-b.x, y-b.y, z-b.z}; }
16    pt3 operator*(T b) {return {x * b, y * b, z * b}; }
17    pt3 operator/(T b) {return {x / b, y / b, z / b}; }
18    bool operator==(pt3 b){return x==b.x&&y==b.y&&z==b.z;}
19
20    T operator*(pt3 b) {return x*b.x+y*b.y+z*b.z;}
21    pt3 operator^(pt3 b) {
22        return pt3{y * b.z - z * b.y,

```

```

23         z * b.x - x * b.z,
24         x * b.y - y * b.x);
25     }
26     friend T abs2(pt3 b) {return b * b; }
27     friend T len (pt3 b) {return sqrt(abs2(b)); }
28     friend ostream& operator<<(ostream& os, pt3 p) {
29         return os << "(" << p.x << ", " <<
30             p.y << ", " << p.z << ")";
31     }
32 };
33
34 template<typename T>
35 struct face {
36     int a, b, c; // 三角形在 vector 裡面的 index
37     pt3<T> q;    // 面積向量 ( 朝外 )
38 };
39
40 /// 警告 ; v 在過程中可能被修改 · 回傳的 face 以修改後的為準
41 ///  $O(n^2)$  · 最多只有  $2n-4$  個面
42 /// 當凸包退化時會回傳空的凸包 · 否則回傳凸包上的每個面
43 template<typename T>
44 vector<face<T>> hull3(vector<pt3<T>> &v) {
45     int n = v.size();
46     if (n < 3) return {};
47     // don't use "==" when you use ld
48     sort(all(v), [&](pt3<T> &p, pt3<T> &q) {
49         return sign(p.x - q.x) ? (p.x < q.x) :
50             (sign(p.y - q.y) ? p.y < q.y : p.z < q.z);
51     });
52     v.resize(unique(v.begin(), v.end()) - v.begin());
53     for (int i = 2; i <= n; ++i) {
54         if (i == n) return {};
55         if (sign(len((v[1] - v[0]) ^ (v[i] - v[0])))) {
56             swap(v[2], v[i]);
57             break;
58         }
59     }
60     pt3<T> tmp_q = (v[1] - v[0]) ^ (v[2] - v[0]);
61     for (int i = 3; i <= n; ++i) {
62         if (i == n) return {};
63         if (sign((v[i] - v[0]) * tmp_q)) {
64             swap(v[3], v[i]);
65             break;
66         }
67     }
68
69     vector<face<T>> f;
70     vector<vector<int>> dead(n, vector<int>(n, true));
71     auto add_face = [&](int a, int b, int c) {
72         f.emplace_back(a, b, c, (v[b] - v[a]) ^ (v[c] - v[a]));
73         dead[a][b] = dead[b][c] = dead[c][a] = false;
74     };
75     add_face(0, 1, 2);
76     add_face(0, 2, 1);
77
78     for (int i = 3; i < n; ++i) {
79         vector<face<T>> f2;
80         for (auto &[a, b, c, q] : f) {
81             if (sign((v[i] - v[a]) * q) > 0)
82                 dead[a][b] = dead[b][c] = dead[c][a] = true;
83             else f2.emplace_back(a, b, c, q);
84         }
85         f.clear();
86         for (face<T> &F : f2) {

```

```

87         int arr[3] = {F.a, F.b, F.c};
88         for (int j = 0; j < 3; ++j) {
89             int a = arr[j], b = arr[(j + 1) % 3];
90             if (dead[b][a]) add_face(b, a, i);
91         }
92     }
93     f.insert(f.end(), all(f2));
94 }
95 return f;
96 } // 15ef50

```

5.29 Pick's Theorem

給定頂點坐標均是整點的簡單多邊形 · 面積 = 內部格點數 + 邊上格點數/2 - 1

6 Graph

6.1 SAT [5a6317]

```

1 struct TWO_SAT {
2     int n, N;
3     vector<vector<int>> G, rev_G;
4     deque<bool> used;
5     vector<int> order, comp;
6     deque<bool> assignment;
7     void init(int _n) {
8         n = _n;
9         N = _n * 2;
10        G.resize(N + 5);
11        rev_G.resize(N + 5);
12    }
13    void dfs1(int v) {
14        used[v] = true;
15        for (int u : G[v]) {
16            if (!used[u])
17                dfs1(u);
18        }
19        order.push_back(v);
20    }
21    void dfs2(int v, int c1) {
22        comp[v] = c1;
23        for (int u : rev_G[v]) {
24            if (comp[u] == -1)
25                dfs2(u, c1);
26        }
27    }
28    bool solve() {
29        order.clear();
30        used.assign(N, false);
31        for (int i = 0; i < N; ++i) {
32            if (!used[i])
33                dfs1(i);
34        }
35        comp.assign(N, -1);
36        for (int i = 0, j = 0; i < N; ++i) {
37            int v = order[N - i - 1];
38            if (comp[v] == -1)
39                dfs2(v, j++);
40        }
41        assignment.assign(n, false);
42        for (int i = 0; i < N; i += 2) {
43            if (comp[i] == comp[i + 1])
44                return false;
45            assignment[i / 2] = (comp[i] > comp[i + 1]);
46        }
47        return true;

```

```

48    }
49    // A or B 都是 0-based
50    void add_disjunction(int a, bool na, int b, bool nb) {
51        // na is true => ~a, na is false => a
52        // nb is true => ~b, nb is false => b
53        a = 2 * a ^ na;
54        b = 2 * b ^ nb;
55        int neg_a = a ^ 1;
56        int neg_b = b ^ 1;
57        G[neg_a].push_back(b);
58        G[neg_b].push_back(a);
59        rev_G[b].push_back(neg_a);
60        rev_G[a].push_back(neg_b);
61        return;
62    }
63    void get_result(vector<int>& res) {
64        res.clear();
65        for (int i = 0; i < n; i++)
66            res.push_back(assignment[i]);
67    }
68 };

```

6.2 Augment Path [f8a5dd]

```

1 struct AugmentPath{
2     int n, m;
3     vector<vector<int>> G;
4     vector<int> mx, my;
5     vector<int> visx, visy;
6     int stamp;
7
8     AugmentPath(int _n, int _m) : n(_n), m(_m), G(n), mx(n,
9         -1), my(m, -1), visx(n), visy(n){
10         stamp = 0;
11     }
12
13     void add(int x, int y){
14         G[x].push_back(y);
15     }
16
17     // bb03e2
18     bool dfs1(int now){
19         visx[now] = stamp;
20
21         for (auto x : G[now]){
22             if (my[x]==-1){
23                 mx[now] = x;
24                 my[x] = now;
25                 return true;
26             }
27         }
28         for (auto x : G[now]){
29             if (visx[my[x]]!=stamp && dfs1(my[x])){
30                 mx[now] = x;
31                 my[x] = now;
32                 return true;
33             }
34         }
35         return false;
36     }
37
38     vector<pair<int, int>> find_max_matching(){
39         vector<pair<int, int>> ret;
40
41         while (true){

```

```

41     stamp++;
42     int tmp = 0;
43     for (int i=0 ; i<n ; i++){
44         if (mx[i]==-1 && dfs1(i)) tmp++;
45     }
46     if (tmp==0) break;
47 }
48
49 for (int i=0 ; i<n ; i++){
50     if (mx[i]!=-1){
51         ret.push_back({i, mx[i]});
52     }
53 }
54 return ret;
55 }
56
57 // 645577
58 void dfs2(int now){
59     visx[now] = true;
60
61     for (auto x : G[now]){
62         if (my[x]!=-1 && visy[x]==false){
63             visy[x] = true;
64             dfs2(my[x]);
65         }
66     }
67 }
68
69 // 要先執行 find_max_matching 一次
70 vector<pair<int, int>> find_min_vertex_cover(){
71     fill(visx.begin(), visx.end(), false);
72     fill(visy.begin(), visy.end(), false);
73
74     vector<pair<int, int>> ret;
75     for (int i=0 ; i<n ; i++){
76         if (mx[i]==-1) dfs2(i);
77     }
78
79     for (int i=0 ; i<n ; i++){
80         if (visx[i]==false) ret.push_back({1, i});
81     }
82     for (int i=0 ; i<m ; i++){
83         if (visy[i]==true) ret.push_back({2, i});
84     }
85
86     return ret;
87 }
88 };

```

6.3 C3C4 [875ea3]

```

1 // 0-based
2 void C3C4(vector<int> deg, vector<array<int, 2>> edges){
3     int N = deg.size();
4     int M = edges.size();
5
6     vector<int> ord(N), rk(N);
7     iota(ord.begin(), ord.end(), 0);
8     sort(ord.begin(), ord.end(), [&](int x, int y) { return
9         deg[x] > deg[y]; });
10    for (int i=0 ; i<N ; i++) rk[ord[i]] = i;
11
12    vector<vector<int>> D(N), adj(N);
13    for (auto [u, v] : edges) {
14        if (rk[u] > rk[v]) swap(u, v);

```

```

14        D[u].emplace_back(v);
15        adj[u].emplace_back(v);
16        adj[v].emplace_back(u);
17    }
18
19    vector<int> vis(N);
20
21    int c3 = 0, c4 = 0;
22    for (int x : ord) { // c3
23        for (int y : D[x]) vis[y] = 1;
24        for (int y : D[x]) for (int z : D[y]){
25            c3 += vis[z]; // xyz is C3
26        }
27        for (int y : D[x]) vis[y] = 0;
28    }
29    for (int x : ord) { // c4
30        for (int y : D[x]) for (int z : adj[y])
31            if (rk[z] > rk[x]) c4 += vis[z]++;
32        for (int y : D[x]) for (int z : adj[y])
33            if (rk[z] > rk[x]) --vis[z];
34    } // both are O(M*sqrt(M)), test @ 2022 CCPC guangzhou
35    cout << c4 << "\n";
36 }

```

6.4 Dinic [961b34]

```

1 // 一般圖：O(EV^2)
2 // 二分圖：O(EVV)
3 struct Flow{
4     using T = int; // 可以換成別的类型
5     struct Edge{
6         int v; T rc; int rid;
7     };
8     vector<vector<Edge>> G;
9     void add(int u, int v, T c){
10         G[u].push_back({v, c, G[v].size()});
11         G[v].push_back({u, 0, G[u].size()-1});
12     }
13     vector<int> dis, it;
14
15     Flow(int n){
16         G.resize(n);
17         dis.resize(n);
18         it.resize(n);
19     }
20
21     // ce56d6
22     T dfs(int u, int t, T f){
23         if (u == t || f == 0) return f;
24         for (int &i=it[u] ; i<G[u].size() ; i++){
25             auto &[v, rc, rid] = G[u][i];
26             if (dis[v]!=dis[u]+1) continue;
27             T df = dfs(v, t, min(f, rc));
28             if (df <= 0) continue;
29             rc -= df;
30             G[v][rid].rc += df;
31             return df;
32         }
33         return 0;
34     }
35
36     // e22e39
37     T flow(int s, int t){
38         T ans = 0;
39         while (true){

```

```

40             fill(dis.begin(), dis.end(), INF);
41             queue<int> q;
42             q.push(s);
43             dis[s] = 0;
44
45             while (q.size()){
46                 int u = q.front(); q.pop();
47                 for (auto [v, rc, rid] : G[u]){
48                     if (rc <= 0 || dis[v] < INF) continue;
49                     dis[v] = dis[u] + 1;
50                     q.push(v);
51                 }
52             }
53             if (dis[t]==INF) break;
54
55             fill(it.begin(), it.end(), 0);
56             while (true){
57                 T df = dfs(s, t, INF);
58                 if (df <= 0) break;
59                 ans += df;
60             }
61             return ans;
62         }
63     }
64
65     // the code below constructs minimum cut
66     void dfs_mincut(int now, vector<bool> &vis){
67         vis[now] = true;
68         for (auto &[v, rc, rid] : G[now]){
69             if (vis[v] == false && rc > 0){
70                 dfs_mincut(v, vis);
71             }
72         }
73     }
74
75     vector<pair<int, int>> construct(int n, int s, vector<
76         pair<int, int>> &E){
77         // E is G without capacity
78         vector<bool> vis(n);
79         dfs_mincut(s, vis);
80         vector<pair<int, int>> ret;
81         for (auto &[u, v] : E){
82             if (vis[u] == true && vis[v] == false){
83                 ret.emplace_back(u, v);
84             }
85         }
86         return ret;
87     }
88 };

```

6.5 Dominator Tree [52b249]

```

1 // 全部都是 0-based
2 // G 要是有向無權圖
3 // 一開始要初始化 G(N, root)
4 // 用完之後要 build
5 // G[i] = 從 root 走到 i 時，一定要走到的點且離 i 最近
6 struct DominatorTree{
7     int N;
8     vector<vector<int>> G;
9     vector<vector<int>> buckets, rg;
10    // dfn[x] = the DFS order of x
11    // rev[x] = the vertex with DFS order x
12    // par[x] = the parent of x
13    vector<int> dfn, rev, par;

```

```

14 vector<int> sdom, dom, idom;
15 vector<int> fa, val;
16 int stamp;
17 int root;
18
19 int operator [] (int x){
20     return idom[x];
21 }
22
23 DominatorTree(int _N, int _root) :
24     N(_N),
25     G(N), buckets(N), rg(N),
26     dfn(N, -1), rev(N, -1), par(N, -1),
27     sdom(N, -1), dom(N, -1), idom(N, -1),
28     fa(N, -1), val(N, -1)
29 {
30     stamp = 0;
31     root = _root;
32 }
33
34 void add_edge(int u, int v){
35     G[u].push_back(v);
36 }
37
38 void dfs(int x){
39     rev[dfn[x] = stamp] = x;
40     fa[stamp] = sdom[stamp] = val[stamp] = stamp;
41     stamp++;
42
43     for (int u : G[x]){
44         if (dfn[u]==-1){
45             dfs(u);
46             par[dfn[u]] = dfn[x];
47         }
48         rg[dfn[u]].push_back(dfn[x]);
49     }
50 }
51
52 int eval(int x, bool first){
53     if (fa[x]==x) return !first ? -1 : x;
54     int p = eval(fa[x], false);
55
56     if (p==-1) return x;
57     if (sdom[val[x]]>sdom[val[fa[x]]]) val[x] = val[fa[x]];
58     fa[x] = p;
59
60     return !first ? p : val[x];
61 }
62
63 void link(int x, int y){
64     fa[x] = y;
65 }
66
67 void build(){
68     dfs(root);
69
70     for (int x=stamp-1 ; x>=0 ; x--){
71         for (int y : rg[x]){
72             sdom[x] = min(sdom[x], sdom[eval(y, true)]);
73         }
74         if (x>0) buckets[sdom[x]].push_back(x);
75         for (int u : buckets[x]){
76             int p = eval(u, true);
77             if (sdom[p]==x) dom[u] = x;
78             else dom[u] = p;

```

```

79     }
80     if (x>0) link(x, par[x]);
81 }
82
83 idom[root] = root;
84 for (int x=1 ; x<stamp ; x++){
85     if (sdom[x]!=dom[x]) dom[x] = dom[dom[x]];
86 }
87 for (int i=1 ; i<stamp ; i++) idom[rev[i]] = rev[dom[i]];
88 }
89 };

```

6.6 EdgeBCC [d09eb1]

```

1 // d09eb1
2 // 0-based · 支援重邊
3 struct EdgeBCC{
4     int n, m, dep, sz;
5     vector<vector<pair<int, int>>> G;
6     vector<vector<int>>> bcc;
7     vector<int> dfn, low, stk, isBridge, bccId;
8     vector<pair<int, int>> edge, bridge;
9
10    EdgeBCC(int _n) : n(_n), m(0), sz(0), dfn(n), low(n), G(n), bcc(n), bccId(n) {}
11
12    void add_edge(int u, int v) {
13        edge.push_back({u, v});
14        G[u].push_back({v, m});
15        G[v].push_back({u, m++});
16    }
17
18    void dfs(int now, int pre) {
19        dfn[now] = low[now] = ++dep;
20        stk.push_back(now);
21
22        for (auto [x, id] : G[now]){
23            if (!dfn[x]){
24                dfs(x, id);
25                low[now] = min(low[now], low[x]);
26            } else if (id!=pre){
27                low[now] = min(low[now], dfn[x]);
28            }
29        }
30
31        if (low[now]==dfn[now]){
32            if (pre!=-1) isBridge[pre] = true;
33            int u;
34            do{
35                u = stk.back();
36                stk.pop_back();
37                bcc[sz].push_back(u);
38                bccId[u] = sz;
39            } while (u!=now);
40            sz++;
41        }
42    }
43
44    void get_bcc() {
45        isBridge.assign(m, 0);
46        dep = 0;
47        for (int i=0 ; i<n ; i++){
48            if (!dfn[i]) dfs(i, -1);
49        }

```

```

50
51     for (int i=0 ; i<m ; i++){
52         if (isBridge[i]){
53             bridge.push_back({edge[i].first , edge[i].second});
54         }
55     }
56 }
57 };

```

6.7 EnumeratePlanarFace [8df3e2]

```

1 struct PlanarGraph { // 0-based index
2     using T = int;
3     int n, m, faces;
4     vector<point<T>> v;
5     vector<vector<pair<int, int>>> D, G;
6     vector<vector<int>>> F;
7     /* D : 對偶圖 · 平面圖的邊的編號是 [0, m - 1],
8        outer face 的編號是 m, face 的編號是 [m + 1, m + faces]
9        F : 存每個 inner face 的 point index */
10    vector<int> conv, nxt, vis;
11
12    PlanarGraph(int n, vector<point<T>> _v) :
13        n(n), m(0), faces(0), v(_v), G(n) {}
14
15    void add_edge(int x, int y) {
16        G[x].emplace_back(y, 2 * m);
17        G[y].emplace_back(x, 2 * m + 1);
18        conv.push_back(x);
19        conv.push_back(y);
20        m++;
21    }
22
23    vector<T> enumerate_face() {
24        D.resize(m + 1);
25        F.resize(m + 1);
26        nxt.resize(2 * m);
27        vis.resize(2 * m);
28        for (int i = 0; i < n; i++) {
29            sort(G[i].begin(), G[i].end(), [&](auto& a, auto& b) {
30                return (v[a.first]-v[i]) < (v[b.first]-v[i]);
31            }); // 極角排序用的 '<' operator
32            int sz = G[i].size(), pre = sz - 1;
33            for (int j = 0; j < sz; pre = j, j++) {
34                nxt[G[i][pre].second] = G[i][j].second ^ 1;
35            }
36        }
37
38        vector<T> ret;
39        // 這個 for 迴圈的 hash value 是 fa2180
40        for (int i = 0; i < 2 * m; i++) if (vis[i] == false){
41            vector<int> pt, outdeg;
42            for (int now = i; !vis[now]; now = nxt[now]) {
43                vis[now] = true;
44                pt.push_back(conv[now]);
45                outdeg.push_back(now / 2);
46            }
47
48            T area = -(v[pt.back()] ^ v[pt.front()]);
49            for (int j = 0; j + 1 < pt.size(); j++) {
50                area -= (v[pt[j]] ^ v[pt[j + 1]]);
51            }
52        }
53    }

```

```

54     if (area > 0) { // inner face
55         F.push_back(move(pt));
56         ret.push_back(area);
57         faces += 1;
58         D.emplace_back();
59         for (auto &u:outdeg) {
60             D[m + faces].emplace_back(u, 0);
61             D[u].emplace_back(m + faces, 1);
62         }
63     }
64     else { // outer face or a tree
65         for (auto &u:outdeg) {
66             D[m].emplace_back(u, 0);
67         }
68     }
69 }
70 return ret;
71 }
72 };

```

6.8 Ford Fulkerson [5bc854]

```

1  const int maxn = 1e5 + 10, INF = 1e9;
2  const long long INF64 = 1e18;
3  struct edge{ int to, cap, rev; };
4  vector<edge> G[maxn];
5  int n, m, s, t, a, b, c;
6  bool vis[maxn];
7  int dfs(int v, int t, int f) {
8      // cout << v << ' ' << t << ' ' << f << '\n';
9      if (v == t) return f;
10     vis[v] = true;
11     for (edge &e: G[v]) {
12         if (!vis[e.to] && e.cap > 0) {
13             int d = dfs(e.to, t, min(f, e.cap));
14             if (d > 0) {
15                 e.cap -= d, G[e.to][e.rev].cap += d;
16                 return d;
17             }
18         }
19     }
20     return 0;
21 }
22 int ford_fulkerson(int s, int t) {
23     int flow = 0, f;
24     // for (int i = 0; i < n; i++) {
25     //     cout << i << " : ";
26     //     for (edge e: G[i])
27     //         cout << '(' << e.to << ', ' << e.cap << ')' <<
28     //             ' ';
29     //     cout << '\n';
30     // }
31     do {
32         memset(vis, false, sizeof(vis));
33         f = dfs(s, t, INF);
34         // for (int i = 0; i < n; i++) {
35         //     cout << i << " : ";
36         //     for (edge e: G[i])
37         //         cout << '(' << e.to << ', ' << e.cap << ')' <<
38         //             ' ';
39         //     cout << '\n';
40         // }
41         // cout << f << '\n';
42         flow += f;
43     } while (f > 0);

```

```

42     return flow;
43 }
44 void init(int n) {
45     for (int i = 0; i < n; i++) G[i].clear();
46 }
47 int main() {
48     cin >> n >> m >> s >> t;
49     init(n);
50     while (m--) {
51         cin >> a >> b >> c;
52         G[a].push_back((edge){b, c, (int)G[b].size()});
53         G[b].push_back((edge){a, 0, (int)G[a].size() - 1});
54     }
55     cout << ford_fulkerson(s, t) << '\n';
56     return 0;
57 }

```

6.9 HLD [f57ec6]

```

1  #include <bits/stdc++.h>
2  #define int long long
3  using namespace std;
4
5  const int N = 100005;
6  vector<int> G[N];
7  struct HLD {
8      vector<int> pa, sz, depth, mxson, topf, id;
9      int n, idcnt = 0;
10     HLD(int _n) : n(_n), pa(_n + 1), sz(_n + 1), depth(_n + 1), mxson(_n + 1), topf(_n + 1), id(_n + 1) {}
11     void dfs1(int v = 1, int p = -1) {
12         pa[v] = p; sz[v] = 1; mxson[v] = 0;
13         depth[v] = (p == -1 ? 0 : depth[p] + 1);
14         for (int u : G[v]) {
15             if (u == p) continue;
16             dfs1(u, v);
17             sz[v] += sz[u];
18             if (sz[u] > sz[mxson[v]]) mxson[v] = u;
19         }
20     }
21     void dfs2(int v = 1, int top = 1) {
22         id[v] = ++idcnt;
23         topf[v] = top;
24         if (mxson[v]) dfs2(mxson[v], top);
25         for (int u : G[v]) {
26             if (u == mxson[v] || u == pa[v]) continue;
27             dfs2(u, u);
28         }
29     }
30     // query 為區間資料結構
31     int path_query(int a, int b) {
32         int res = 0;
33         while (topf[a] != topf[b]) { /// 若不在同一條鍊上
34             if (depth[topf[a]] < depth[topf[b]]) swap(a, b);
35             res = max(res, 0ll); // query : l = id[topf[a]],
36                                 r = id[a]
37             a = pa[topf[a]];
38         }
39         /// 此時已在同一條鍊上
40         if (depth[a] < depth[b]) swap(a, b);
41         res = max(res, 0ll); // query : l = id[b], r = id[a]
42         return res;
43     }
44 }

```

6.10 Kosaraju SCC [c7d5aa]

```

1  // 給定一個有向圖，迴傳傳縮點後的圖、SCC 的資訊
2  // 所有點都以 based-0 編號
3  // 函式：
4  // .compress: O(n Log n) 計算 G3、SCC、SCC_id 的資訊，並把縮
5  // 點後的結果存在 result 裡
6  // SCC_id[i] = 第 i 個點在第幾個 SCC
7  struct SCC_compress{
8      int N, M, sz;
9      vector<vector<int>> G, inv_G, result;
10     vector<pair<int, int>> edges;
11     vector<bool> vis;
12     vector<int> order;
13
14     vector<vector<int>> SCC;
15     vector<int> SCC_id;
16
17     SCC_compress(int _N) :
18         N(_N), M(0), sz(0),
19         G(N), inv_G(N),
20         vis(N), SCC_id(N)
21     {}
22
23     vector<int> operator [] (int x){
24         return result[x];
25     }
26
27     void add_edge(int u, int v){
28         G[u].push_back(v);
29         inv_G[v].push_back(u);
30         edges.push_back({u, v});
31         M++;
32     }
33
34     void dfs1(vector<vector<int>> &G, int now){
35         vis[now] = 1;
36         for (auto x : G[now]) if (!vis[x]) dfs1(G, x);
37         order.push_back(now);
38     }
39
40     void dfs2(vector<vector<int>> &G, int now){
41         SCC_id[now] = SCC.size() - 1;
42         SCC.back().push_back(now);
43         vis[now] = 1;
44         for (auto x : G[now]) if (!vis[x]) dfs2(G, x);
45     }
46
47     void compress(){
48         fill(vis.begin(), vis.end(), 0);
49         for (int i=0; i<N; i++) if (!vis[i]) dfs1(G, i);
50
51         fill(vis.begin(), vis.end(), 0);
52         reverse(order.begin(), order.end());
53         for (int i=0; i<N; i++){
54             if (!vis[order[i]]){
55                 SCC.push_back(vector<int>());
56                 dfs2(inv_G, order[i]);
57             }
58         }
59
60         result.resize(SCC.size());
61         sz = SCC.size();
62         for (auto [u, v] : edges){

```

```

63     if (SCC_id[u]!=SCC_id[v]) result[SCC_id[u]].
        push_back(SCC_id[v]);
64     }
65     for (int i=0 ; i<SCC.size() ; i++){
66         sort(result[i].begin(), result[i].end());
67         result[i].resize(unique(result[i].begin(), result
            [i].end())-result[i].begin());
68     }
69 }
70 };

```

6.11 Kuhn Munkres [e66c35]

```

1 // O(n^3) 找到最大權匹配
2 struct KuhnMunkres{
3     int n; // max(n, m)
4     vector<vector<int>> G;
5     vector<int> match, lx, ly, visx, visy;
6     vector<int> slack;
7     int stamp = 0;
8
9     KuhnMunkres(int n) : n(n), G(n, vector<int>(n)), lx(n),
        ly(n), slack(n), match(n), visx(n), visy(n) {}
10
11     void add(int x, int y, int w){
12         G[x][y] = max(G[x][y], w);
13     }
14
15     bool dfs(int i, bool aug){ // aug = true 表示要更新 match
16         if (visx[i]==stamp) return false;
17         visx[i] = stamp;
18
19         for (int j=0 ; j<n ; j++){
20             if (visy[j]==stamp) continue;
21             int d = lx[i]+ly[j]-G[i][j];
22
23             if (d==0){
24                 visy[j] = stamp;
25                 if (match[j]==-1 || dfs(match[j], aug)){
26                     if (aug){
27                         match[j] = i;
28                     }
29                     return true;
30                 }
31             }else{
32                 slack[j] = min(slack[j], d);
33             }
34         }
35         return false;
36     }
37
38     bool augment(){
39         for (int j=0 ; j<n ; j++){
40             if (visy[j]!=stamp && slack[j]==0){
41                 visy[j] = stamp;
42                 if (match[j]==-1 || dfs(match[j], false)){
43                     return true;
44                 }
45             }
46         }
47         return false;
48     }
49
50     void relabel(){
51         int delta = INF;

```

```

52     for (int j=0 ; j<n ; j++){
53         if (visy[j]!=stamp) delta = min(delta, slack[j]);
54     }
55     for (int i=0 ; i<n ; i++){
56         if (visx[i]==stamp) lx[i] -= delta;
57     }
58     for (int j=0 ; j<n ; j++){
59         if (visy[j]==stamp) ly[j] += delta;
60         else slack[j] -= delta;
61     }
62 }
63
64 int solve(){
65
66     for (int i=0 ; i<n ; i++){
67         lx[i] = 0;
68         for (int j=0 ; j<n ; j++){
69             lx[i] = max(lx[i], G[i][j]);
70         }
71     }
72
73     fill(ly.begin(), ly.end(), 0);
74     fill(match.begin(), match.end(), -1);
75
76     for(int i = 0; i < n; i++) {
77         fill(slack.begin(), slack.end(), INF);
78         stamp++;
79         if(dfs(i, true)) continue;
80
81         while(augment()==false) relabel();
82         stamp++;
83         dfs(i, true);
84     }
85
86     int ans = 0;
87     for (int j=0 ; j<n ; j++){
88         if (match[j]!=-1){
89             ans += G[match[j]][j];
90         }
91     }
92     return ans;
93 }
94 };

```

6.12 LCA [5b6a5b]

```

1 // 1-based · 可以支援森林 · 0 是超級源點 · 所有樹都要跟他建邊
2 struct Tree{
3     int N, M = 0, H;
4     vector<int> parent, dep;
5     vector<vector<int>> G, LCA;
6
7     Tree(int _N) : N(_N+1), H(__lg(N)+1), parent(N, -1), dep(
        N), G(N){
8         LCA.resize(H, vector<int>(N, 0));
9     }
10
11     void add_edge(int u, int v){
12         M++;
13         G[u].push_back(v);
14         G[v].push_back(u);
15     }
16
17     void dfs(int now = 0, int pre = 0){
18         dep[now] = dep[pre]+1;

```

```

19         parent[now] = pre;
20         for (auto x : G[now]){
21             if (x==pre) continue;
22             dfs(x, now);
23         }
24     }
25
26     void build_LCA(int root = 0){
27         dfs();
28         for (int i=0 ; i<N ; i++) LCA[0][i] = parent[i];
29         for (int i=1 ; i<H ; i++){
30             for (int j=0 ; j<N ; j++){
31                 LCA[i][j] = LCA[i-1][LCA[i-1][j]];
32             }
33         }
34
35         int jump(int u, int step){
36             for (int i=0 ; i<H ; i++){
37                 if (step&(1<<i)) u = LCA[i][u];
38             }
39             return u;
40         }
41
42         int get_LCA(int u, int v){
43             if (dep[u]<dep[v]) swap(u, v);
44             u = jump(u, dep[u]-dep[v]);
45             if (u==v) return u;
46             for (int i=H-1 ; i>=0 ; i--){
47                 if (LCA[i][u]!=LCA[i][v]){
48                     u = LCA[i][u];
49                     v = LCA[i][v];
50                 }
51             }
52             return parent[u];
53         }
54     }
55 };

```

6.13 MCMF [0d524d]

```

1 struct Flow {
2     struct Edge {
3         int u, rc, k, rv;
4     };
5
6     vector<vector<Edge>> G;
7     vector<int> par, par_eid;
8     Flow(int n) : G(n+1), par(n+1), par_eid(n+1) {}
9
10     // v->u, capacity: c, cost: k
11     void add(int v, int u, int c, int k){
12         G[v].push_back({u, c, k, G[u].size()});
13         G[u].push_back({v, 0, -k, G[v].size()-1});
14     }
15
16     // 6d1140
17     int spfa(int s, int t){
18         fill(par.begin(), par.end(), -1);
19         vector<int> dis(par.size(), INF);
20         vector<bool> in_q(par.size(), false);
21         queue<int> Q;
22         dis[s] = 0;
23         in_q[s] = true;
24         Q.push(s);
25

```

```

26 while (!Q.empty()){
27     int v = Q.front();
28     Q.pop();
29     in_q[v] = false;
30
31     for (int i=0; i<G[v].size(); i++){
32         auto [u, rc, k, rv] = G[v][i];
33         if (rc>0 && dis[v]+k<dis[u]){
34             dis[u] = dis[v]+k;
35             par[u] = v;
36             par_eid[u] = i;
37             if (!in_q[u]) Q.push(u);
38             in_q[u] = true;
39         }
40     }
41 }
42
43 return dis[t];
44 }
45
46 // return <max flow, min cost>, d7e7ad
47 pair<int, int> flow(int s, int t){
48     int fl = 0, cost = 0, d;
49     while ((d = spfa(s, t))<INF){
50         int cur = INF;
51         for (int v=t; v!=s; v=par[v])
52             cur = min(cur, G[par[v]][par_eid[v]].rc);
53         fl += cur;
54         cost += d*cur;
55         for (int v=t; v!=s; v=par[v]){
56             G[par[v]][par_eid[v]].rc -= cur;
57             G[v][G[par[v]][par_eid[v]].rv].rc += cur;
58         }
59     }
60     return {fl, cost};
61 }
62 };

```

6.14 Tarjan SCC [8b2350]

```

1 struct tarjan_SCC {
2     int now_T, now_SCCs;
3     vector<int> dfn, low, SCC;
4     stack<int> S;
5     vector<vector<int>> E;
6     vector<bool> vis, in_stack;
7
8     tarjan_SCC(int n) {
9         init(n);
10    }
11    void init(int n) {
12        now_T = now_SCCs = 0;
13        dfn = low = SCC = vector<int>(n);
14        E = vector<vector<int>>(n);
15        S = stack<int>();
16        vis = in_stack = vector<bool>(n);
17    }
18    void add(int u, int v) {
19        E[u].push_back(v);
20    }
21    void build() {
22        for (int i = 0; i < dfn.size(); ++i) {
23            if (!dfn[i]) dfs(i);
24        }
25    }

```

```

26 void dfs(int v) {
27     now_T++;
28     vis[v] = in_stack[v] = true;
29     dfn[v] = low[v] = now_T;
30     S.push(v);
31     for (auto &i:E[v]) {
32         if (!vis[i]) {
33             vis[i] = true;
34             dfs(i);
35             low[v] = min(low[v], low[i]);
36         }
37         else if (in_stack[i]) {
38             low[v] = min(low[v], dfn[i]);
39         }
40     }
41     if (low[v] == dfn[v]) {
42         int tmp;
43         do {
44             tmp = S.top();
45             S.pop();
46             SCC[tmp] = now_SCCs;
47             in_stack[tmp] = false;
48         } while (tmp != v);
49         now_SCCs += 1;
50     }
51 }
52 };

```

6.15 Theorem

- 任意圖
 - 最大匹配 + 最小邊覆蓋 = n (不能有孤點)
 - 點覆蓋的補集是獨立集。最小點覆蓋 + 最大獨立集 = n
 - w (最小點覆蓋) + w (最大權獨立集) = $\sum w_v$
 - (帶點權的二分圖可以用最小割解。構造請參考 Augment Path.cpp)
- 二分圖
 - 最小點覆蓋 = 最大匹配 = n - 最大獨立集

- 只有邊帶權的二分圖

- w-vertex-cover (帶權點覆蓋): 每條邊的兩個連接點被選中的次數總和至少要是 w_e 。
- w-weight matching (帶權匹配)
- minimum vertex count of w-vertex-cover = maximum weight count of w-weight matching (一個點可以被選很多次。但邊不行)

- 點、邊都帶權的二分圖的定理

- b-matching: 假設 v 的點權是 b_v 。那所有 v 的匹配邊 e 的權重都要滿足 $\sum w_e \leq b_v$ 。
- The maximum w-weight of a b-matching equals the minimum b-weight of vertices in a w-vertex-cover.

6.16 Tree Isomorphism [12a4ed]

```

1 // Please change "mul" to a random number (> N) before submit
2 struct Tree_Hash { // 1-based
3     using T = __int128;
4     const T mod = (T)(1e18 + 3), mul = 435216;
5     vector<vector<int>> E;

```

```

6     Tree_Hash(int _n) : E(_n + 1) {}
7     void add_edge(int u, int v) {
8         E[u].push_back(v); E[v].push_back(u);
9     }
10    pair<T, T> h(int v, int p = -1) {
11        vector<pair<T, T>> s;
12        for (auto &i:E[v]) if (i != p) {
13            s.push_back(h(i, v));
14        }
15        T res = s.size() + 1, prod = mul;
16        sort(s.begin(), s.end());
17        for (auto &[i, j] : s) {
18            res = (res + i * prod) % mod;
19            prod = prod * j % mod;
20        }
21        return {res, prod};
22    }
23    T cal(int v = 1) {
24        int n = 0, c1 = -1, c2 = -1;
25        auto dfs = [&](auto&& f, int v, int p) -> int {
26            int sz = 1, mx = 0;
27            for (int u : E[v]) if (u != p) {
28                int s = f(f, u, v);
29                sz += s;
30                mx = max(mx, s);
31            }
32            mx = max(mx, n - sz);
33            if (n && mx * 2 <= n) (~c1 ? c2 : c1) = v;
34            return sz;
35        };
36        n = dfs(dfs, v, -1); dfs(dfs, v, -1);
37        E.push_back({c1, (c2 == -1) ? c1 : c2});
38        return h(E.size() - 1).first;
39    }
40 };

```

6.17 prufer reverse [2f831d]

```

1 // K_n 有  $n^{n-2}$  種生成樹
2 vector<pair<int, int>> prufer_reverse(vector<int> &v) {
3     int n = v.size() + 2, mn = 1, mx = n; // 1-based
4     vector<int> deg(mx + 1);
5     for (auto &i:v) deg[i] += 1;
6     int ind = mn;
7     while (deg[ind] != 0) ind += 1;
8     int leaf = ind;
9     vector<pair<int, int>> res;
10    for (auto &i:v) {
11        res.emplace_back(leaf, i);
12        deg[i] -= 1;
13        if (deg[i] == 0 && i < ind) leaf = i;
14        else {
15            do ind += 1; while (deg[ind] != 0);
16            leaf = ind;
17        }
18    }
19    res.emplace_back(leaf, mx);
20    return res;
21 }

```

6.18 圖方樹 [5a6c9f]

```

1 struct BCC_AP { // 0-based, remember to build, 不支援重邊
2     int n, nbcc; // 注意孤點不會有任何方點連接
3     vector<vector<int>> E, F; // id >= n: 方點
4     vector<int> pa, dep, low, stk, paf, depf;
5     void dfs(int v, int p) {
6         dep[v] = low[v] = ~p ? dep[p] + 1 : 0;
7         stk.push_back(v), pa[v] = p;
8         for (auto& u : E[v]) if (u != p) {
9             if (low[u] == -1) {
10                 dfs(u, v), low[v] = min(low[v], low[u]);
11                 if (low[u] >= dep[v]) {
12                     int id = nbcc++, x;
13                     do {
14                         x = stk.back(), stk.pop_back();
15                         F[id + n].push_back(x), F[x].
16                             push_back(id + n);
17                     } while (x != u);
18                     F[id + n].push_back(v), F[v].push_back(id
19                         + n);
20                 } else low[v] = min(low[v], dep[u]);
21             }
22         }
23         bool is_bridge(int u, int v) {
24             return (F[bcc_id(u, v) + n].size() == 2); }
25         // 判斷原圖上的一個點是不是割點
26         bool is_cut(int x) { return F[x].size() != 1; }
27         // 回傳第 id 個 bcc 的每個點 (0-based)
28         vector<int> bcc(int id) { return F[id + n]; }
29         // 對於邊 (u, v) 回傳包含它的 bcc id (每條邊都恰好被一個
30             bcc 包含)
31         int bcc_id(int u, int v) { // starts from 0
32             return paf[depf[u] < depf[v] ? v : u] - n; }
33         // 計算在新圖上的 depf, paf 陣列·呼叫 bcc_id() 的時候會
34             用到
35         void dfs2(int v, int p) {
36             depf[v] = ~p ? depf[p] + 1 : 0, paf[v] = p;
37             for (int u : F[v]) if (u != p) dfs2(u, v);
38         }
39         void build() {
40             low.assign(n, -1);
41             for (int i = 0; i < n; ++i) if (low[i] == -1)
42                 dfs(i, -1), dfs2(i, -1);
43         }
44         void add_edge(int u, int v) {
45             E[u].push_back(v), E[v].push_back(u);
46         }
47         BCC_AP (int _n) : n(_n), nbcc(0), E(n), F(2 * n), pa(n),
48             dep(n), low(n), stk(), paf(n * 2), depf(n * 2) {}
49     };

```

6.19 最大權閉合圖 [6ca663]

```

1 // 邊 u → v 表示選 u 就要選 v (0-based)
2 // 保證回傳值非負
3 // 構造：呼叫 construct·選那些 vis 是 true 的點。
4 // 一般圖：O(n²m) / 二分圖：O(m√n)
5 template<typename U>
6 U maximum_closure(vector<U> w, vector<pair<int,int>> EV) {
7     int n = w.size(), S = n + 1, T = n + 2;
8     Flow G(T + 5); // Graph/Dinic.cpp

```

```

9     U sum = 0;
10     for (int i = 0; i < n; ++i) {
11         if (w[i] > 0) {
12             G.add(S, i, w[i]);
13             sum += w[i];
14         }
15         else if (w[i] < 0) {
16             G.add(i, T, abs(w[i]));
17         }
18     }
19     for (auto &[u, v] : EV) { // 請務必確保 INF > Σ|w_i|
20         G.add(u, v, INF);
21     }
22     U cut = G.flow(S, T);
23     return sum - cut;
24 }

```

7 Math

7.1 Burnside's Lemma

$$\sum_{k=1}^n \frac{c(k)}{n}$$

- n : 有多少種置換方式 (例如：旋轉方式)

- $c(k)$: 所有可能中·經過 k 次旋轉後·仍不會和別人相同的方式的數量

7.2 CRT [6a6da6]

```

1 using ll = __int128;
2 //a * p.first + b * p.second = gcd(a, b)
3 pair<ll, ll> extgcd(ll a, ll b) {
4     if (b == 0) return {1, 0};
5     auto [y, x] = extgcd(b, a % b);
6     return pair<ll, ll>(x, y - (a / b) * x);
7 }
8
9 pair<ll, ll> CRT(ll x1, ll m1, ll x2, ll m2) {
10     ll g = __gcd(m1, m2);
11     if ((x2 - x1) % g) return make_pair(-1, -1); // no sol
12     m1 /= g, m2 /= g;
13     pair<ll, ll> p = extgcd(m1, m2);
14     ll lcm = m1 * m2 * g;
15     ll res = p.first * (x2 - x1) * m1 + x1;
16     // be careful with overflow
17     return make_pair((res % lcm + lcm) % lcm, lcm);
18 }
19
20 pair<ll, ll> CRT_arr(vector<ll> &a, vector<ll> &m) {
21     pair<ll, ll> res(a[0], m[0]);
22     for (int i = 1; i < a.size(); ++i) {
23         res = CRT(res.first, res.second, a[i], m[i]);
24         if (res.first == -1) break;
25     }
26     return res;
27 }

```

7.3 Catalan Number

任意括號序列： $C_n = \frac{1}{n+1} \binom{2n}{n}$

7.4 Floor Sum [5d1c9e]

```

1 // Returns sum_{i=0}^{n-1} floor((a*i + b) / m)
2 // Log( max(a, m) )
3 int floor_sum(int n, int m, int a, int b) {
4     int ans = 0;
5     while (true) {
6         if (a >= m) {
7             ans += (n - 1) * n * (a / m) / 2;
8             a %= m;
9         }
10        if (b >= m) {
11            ans += n * (b / m);
12            b %= m;
13        }
14        int y_max = a * n + b;
15        if (y_max < m) break;
16        n = y_max / m;
17        b = y_max % m;
18        swap(m, a);
19    }
20    return ans;
21 }

```

7.5 Josephus Problem [e0ed50]

```

1 // 有 n 個人·第偶數個報數的人被刪掉·問第 k 個被踢掉的是誰
2 int solve(int n, int k){
3     if (n==1) return 1;
4     if (k==(n+1)/2){
5         if (2*k>n) return 2*k%n;
6         else return 2*k;
7     }else{
8         int res=solve(n/2, k-(n+1)/2);
9         if (n&1) return 2*res+1;
10        else return 2*res-1;
11    }
12 }

```

7.6 Lagrange any x [1f2c26]

```

1 // init: (x1, y1), (x2, y2) in a vector
2 struct Lagrange{
3     int n;
4     vector<pair<int, int>> v;
5
6     Lagrange(vector<pair<int, int>> &_v){
7         n = _v.size();
8         v = _v;
9     }
10
11     // O(n² Log MAX_A)
12     int solve(int x){
13         int ret = 0;
14         for (int i=0 ; i<n ; i++){
15             int now = v[i].second;
16             for (int j=0 ; j<n ; j++){
17                 if (i==j) continue;
18                 now *= ((x-v[j].first+MOD)%MOD);
19                 now %= MOD;
20                 now *= (qp((v[i].first-v[j].first+MOD)%MOD,
21                     MOD-2)+MOD)%MOD;
22                 now %= MOD;
23             }

```

```

24         ret = (ret+now)%MOD;
25     }
26     return ret;
27 }
28 };

```

7.7 Lagrange continuous x [57536a]

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int MAX_N = 5e5 + 10;
5 const int mod = 1e9 + 7;
6
7 long long inv_fac[MAX_N];
8
9 inline int fp(long long x, int y) {
10     int ret = 1;
11     for (; y; y >>= 1) {
12         ret = (y & 1) ? (ret * x % mod) : ret;
13         x = x * x % mod;
14     }
15     return ret;
16 }
17
18 // TO USE THIS TEMPLATE, YOU MUST MAKE SURE THAT THE MOD
19 // NUMBER IS A PRIME.
20 struct Lagrange {
21     /*
22      * Initialize a polynomial with f(x_0), f(x_0 + 1), ..., f(
23      * x_0 + n).
24      * This determines a polynomial f(x) whose degree is at most
25      * n.
26      * Then you can call sample(x) and you get the value of f(x)
27      *
28      * Complexity of init() and sample() are both O(n).
29      */
30     int m, shift; // m = n + 1
31     vector<int> v, mul;
32     // You can use this function if you don't have inv_fac array
33     // already.
34     void construct_inv_fac() {
35         long long fac = 1;
36         for (int i = 2; i < MAX_N; ++i) {
37             fac = fac * i % mod;
38         }
39         inv_fac[MAX_N - 1] = fp(fac, mod - 2);
40         for (int i = MAX_N - 1; i >= 1; --i) {
41             inv_fac[i - 1] = inv_fac[i] * i % mod;
42         }
43     }
44     // You call init() many times without having a second
45     // instance of this struct.
46     void init(int X_0, vector<int> &u) {
47         v = u;
48         shift = ((1 - X_0) % mod + mod) % mod;
49         if (v.size() == 1) v.push_back(v[0]);
50         m = v.size();
51         mul.resize(m);
52     }
53     // You can use sample(x) instead of sample(x % mod).
54     int sample(int x) {
55         x = ((long long)x + shift) % mod;
56         x = (x < 0) ? (x + mod) : x;
57         long long now = 1;

```

```

52     for (int i = m; i >= 1; --i) {
53         mul[i - 1] = now;
54         now = now * (x - i) % mod;
55     }
56     int ret = 0;
57     bool neg = (m - 1) & 1;
58     now = 1;
59     for (int i = 1; i <= m; ++i) {
60         int up = now * mul[i - 1] % mod;
61         int down = inv_fac[m - i] * inv_fac[i - 1] % mod;
62         int tmp = ((long long)v[i - 1] * up % mod) * down
63             % mod;
64         ret += (neg && tmp) ? (mod - tmp) : (tmp);
65         ret = (ret >= mod) ? (ret - mod) : ret;
66         now = now * (x - i) % mod;
67         neg ^= 1;
68     }
69     return ret;
70 };
71
72 int main() {
73     int n; cin >> n;
74     vector<int> v(n);
75     for (int i = 0; i < n; ++i) {
76         cin >> v[i];
77     }
78     Lagrange L;
79     L.construct_inv_fac();
80     L.init(0, v);
81     int x; cin >> x;
82     cout << L.sample(x);
83 }

```

7.8 Linear Mod Inverse [ecf71e]

```

1 // 線性求 1-based a[i] 對 p 的乘法反元素
2 vector<int> s(n+1, 1), invS(n+1), invA(n+1);
3 for (int i=1; i<=n; i++) s[i] = s[i-1]*a[i]%p;
4 invS[n] = qp(s[n], p-2, p);
5 for (int i=n; i>=1; i--) invS[i-1] = invS[i]*a[i]%p;
6 for (int i=1; i<=n; i++) invA[i] = invS[i]*s[i-1]%p;

```

7.9 Lucas's Theorem [b37dcf]

```

1 // 對於很大的 C^n_{m} 對質數 p 取模，只要 p 不大就可以用。
2 int Lucas(int n, int m, int p){
3     if (m==0) return 1;
4     return (C(n%p, m%p, p)*Lucas(n/p, m/p, p)%p);
5 }

```

7.10 Matrix [8d1a23]

```

1 struct Matrix{
2     int n, m;
3     vector<vector<int>> arr;
4
5     Matrix(int _n, int _m){
6         n = _n;
7         m = _m;
8         arr.assign(n, vector<int>(m));
9     }
10
11     vector<int> & operator [] (int i){

```

```

12         return arr[i];
13     }
14
15     Matrix operator * (Matrix b){
16         Matrix ret(n, b.m);
17         for (int i=0; i<n; i++){
18             for (int j=0; j<b.m; j++){
19                 for (int k=0; k<m; k++){
20                     ret.arr[i][j] += arr[i][k]*b.arr[k][j]%
21                         MOD;
22                     ret.arr[i][j] %= MOD;
23                 }
24             }
25         }
26         return ret;
27     }
28
29     Matrix pow(int p){
30         Matrix ret(n, n), mul = *this;
31         for (int i=0; i<n; i++){
32             ret.arr[i][i] = 1;
33         }
34
35         for (; p; p>>=1){
36             if (p&1) ret = ret*mul;
37             mul = mul*mul;
38         }
39
40         return ret;
41     }
42
43     int det(){
44         vector<vector<int>> arr = this->arr;
45         bool flag = false;
46         for (int i=0; i<n; i++){
47             int target = -1;
48             for (int j=i; j<n; j++){
49                 if (arr[j][i]){
50                     target = j;
51                     break;
52                 }
53             }
54             if (target== -1) return 0;
55             if (i!=target){
56                 swap(arr[i], arr[target]);
57                 flag = !flag;
58             }
59
60             for (int j=i+1; j<n; j++){
61                 if (!arr[j][i]) continue;
62                 int freq = arr[j][i]*qp(arr[i][i], MOD-2)%MOD;
63
64                 for (int k=i; k<n; k++){
65                     arr[j][k] -= freq*arr[i][k];
66                     arr[j][k] = (arr[j][k]%MOD+MOD)%MOD;
67                 }
68             }
69
70             int ret = !flag ? 1 : MOD-1;
71             for (int i=0; i<n; i++){
72                 ret *= arr[i][i];
73                 ret %= MOD;
74             }
75             return ret;

```

```
76 }
77 };
```

7.11 Matrix 01 [8d542a]

```
1 const int MAX_N = (1LL<<12);
2 struct Matrix{
3     int n, m;
4     vector<bitset<MAX_N>> arr;
5
6     Matrix(int _n, int _m){
7         n = _n;
8         m = _m;
9         arr.resize(n);
10    }
11
12    Matrix operator * (Matrix b){
13        Matrix b_t(b.m, b.n);
14        for (int i=0 ; i<b.n ; i++){
15            for (int j=0 ; j<b.m ; j++){
16                b_t.arr[j][i] = b.arr[i][j];
17            }
18        }
19
20        Matrix ret(n, b.m);
21        for (int i=0 ; i<n ; i++){
22            for (int j=0 ; j<b.m ; j++){
23                ret.arr[i][j] = ((arr[i]&b_t.arr[j]).count()
24                    &1);
25            }
26        }
27        return ret;
28    }
29 };
```

7.12 Matrix Tree Theorem

目標：給定一張無向圖，問他的生成樹數量。
方法：先把所有自環刪掉，定義 Q 為以下矩陣

$$Q_{i,j} = \begin{cases} \deg(v_i) & \text{if } i = j \\ -(\text{邊 } v_i v_j \text{ 的數量}) & \text{otherwise} \end{cases}$$

接著刪掉 Q 的第一個 row 跟 column，它的 determinant 就是答案。
目標：給定一張有向圖，問他的以 r 為根，可以走到所有點生成樹數量。

方法：先把所有自環刪掉，定義 Q 為以下矩陣

$$Q_{i,j} = \begin{cases} \deg_{in}(v_i) & \text{if } i = j \\ -(\text{邊 } v_i v_j \text{ 的數量}) & \text{otherwise} \end{cases}$$

接著刪掉 Q 的第 r 個 row 跟 column，它的 determinant 就是答案。

7.13 Miller Rabin [5694bf]

```
1 // O(k Log^3 n), k = llsprp.size()
2 using Uint = unsigned long long;
3 Uint modmul(Uint a, Uint b, Uint m) {
4     int ret = a*b - m*(Uint)((long double)a*b/m);
5     return ret + m*(ret < 0) - m*(ret>=(int)m);
6 }
7
8 int qp(int b, int p, int m){
```

```
9     int ret = 1;
10    for ( ; p ; p>>=1){
11        if (p&1) ret = modmul(ret, b, m);
12        b = modmul(b, b, m);
13    }
14    return ret;
15 }
16
17 // ed23aa
18 vector<int> llsprp = {2, 325, 9375, 28178, 450775, 9780504,
19     1795265022};
20 bool is_prime(int n, vector<int> sprp = llsprp){
21     if (n==2) return 1;
22     if (n<2 || n%2==0) return 0;
23
24     int t = 0;
25     int u = n-1;
26     for ( ; u%2==0 ; t++) u>>=1;
27
28     for (int i=0 ; i<sprp.size() ; i++){
29         int a = sprp[i]%n;
30         if (a==0 || a==1 || a==n-1) continue;
31         int x = qp(a, u, n);
32         if (x==1 || x==n-1) continue;
33         for (int j=0 ; j<t ; j++){
34             x = modmul(x, x, n);
35             if (x==1) return 0;
36             if (x==n-1) break;
37         }
38         if (x==n-1) continue;
39         return false;
40     }
41
42     return true;
43 }
```

7.14 Pollard Rho [a5daef]

```
1 mt19937 seed(chrono::steady_clock::now().time_since_epoch().
2     count());
3 int rnd(int l, int r){
4     return uniform_int_distribution<int>(l, r)(seed);
5 }
6 // O(n^{1/4}) 回傳 1 或自己的因數、記得先判斷 n 是不是質數
7 // (用 Miller-Rabin)
8 // c1670c
9 int Pollard_Rho(int n){
10    int s = 0, t = 0;
11    int c = rnd(1, n-1);
12
13    int step = 0, goal = 1;
14    int val = 1;
15
16    for (goal=1 ; ; goal<=1, s=t, val=1){
17        for (step=1 ; step<=goal ; step++){
18            t = ((__int128)t*t+c)%n;
19            val = ((__int128)val*abs(t-s)%n;
20
21            if ((step % 127) == 0){
22                int d = __gcd(val, n);
23                if (d>1) return d;
24            }
```

```
25    }
26
27    int d = __gcd(val, n);
28    if (d>1) return d;
29 }
30 }
```

7.15 Polynomial [51ca3b]

```
1 struct Poly {
2     int len, deg;
3     int *a;
4     // Len = 2^k >= the original Length
5     Poly(): len(0), deg(0), a(nullptr) {}
6     Poly(int _n) {
7         len = 1;
8         deg = _n - 1;
9         while (len < _n) len <= 1;
10        a = (ll*) calloc(len, sizeof(ll));
11    }
12    Poly(int l, int d, int *b) {
13        len = l;
14        deg = d;
15        a = b;
16    }
17    void resize(int _n) {
18        int len1 = 1;
19        while (len1 < _n) len1 <= 1;
20        int *res = (ll*) calloc(len1, sizeof(ll));
21        for (int i = 0; i < min(len, _n); i++) {
22            res[i] = a[i];
23        }
24        len = len1;
25        deg = _n - 1;
26        free(a);
27        a = res;
28    }
29    Poly& operator=(const Poly rhs) {
30        this->len = rhs.len;
31        this->deg = rhs.deg;
32        this->a = (ll*)realloc(this->a, sizeof(ll) * len);
33        copy(rhs.a, rhs.a + len, this->a);
34        return *this;
35    }
36    Poly operator*(Poly rhs) {
37        int l1 = this->len, l2 = rhs.len;
38        int d1 = this->deg, d2 = rhs.deg;
39        while (l1 > 0 and this->a[l1 - 1] == 0) l1--;
40        while (l2 > 0 and rhs.a[l2 - 1] == 0) l2--;
41        int l = 1;
42        while (l < max(l1 + l2 - 1, d1 + d2 + 1)) l <= 1;
43        int *x, *y, *res;
44        x = (ll*) calloc(l, sizeof(ll));
45        y = (ll*) calloc(l, sizeof(ll));
46        res = (ll*) calloc(l, sizeof(ll));
47        copy(this->a, this->a + l1, x);
48        copy(rhs.a, rhs.a + l2, y);
49        ntt.tran(l, x); ntt.tran(l, y);
50        FOR (i, 0, l - 1)
51            res[i] = x[i] * y[i] % mod;
52        ntt.tran(l, res, true);
53        free(x); free(y);
54        return Poly(l, d1 + d2, res);
55    }
56    Poly operator+(Poly rhs) {
```

```

57     int l1 = this->len, l2 = rhs.len;
58     int l = max(l1, l2);
59     Poly res;
60     res.len = l;
61     res.deg = max(this->deg, rhs.deg);
62     res.a = (ll*) calloc(l, sizeof(ll));
63     FOR (i, 0, l - 1) {
64         res.a[i] += this->a[i];
65         if (res.a[i] >= mod) res.a[i] -= mod;
66     }
67     FOR (i, 0, l2 - 1) {
68         res.a[i] += rhs.a[i];
69         if (res.a[i] >= mod) res.a[i] -= mod;
70     }
71     return res;
72 }
73 Poly operator-(Poly rhs) {
74     int l1 = this->len, l2 = rhs.len;
75     int l = max(l1, l2);
76     Poly res;
77     res.len = l;
78     res.deg = max(this->deg, rhs.deg);
79     res.a = (ll*) calloc(l, sizeof(ll));
80     FOR (i, 0, l1 - 1) {
81         res.a[i] += this->a[i];
82         if (res.a[i] >= mod) res.a[i] -= mod;
83     }
84     FOR (i, 0, l2 - 1) {
85         res.a[i] -= rhs.a[i];
86         if (res.a[i] < 0) res.a[i] += mod;
87     }
88     return res;
89 }
90 Poly operator*(const int rhs) {
91     Poly res;
92     res = *this;
93     FOR (i, 0, res.len - 1) {
94         res.a[i] = res.a[i] * rhs % mod;
95         if (res.a[i] < 0) res.a[i] += mod;
96     }
97     return res;
98 }
99 Poly(vector<int> f) {
100     int _n = f.size();
101     len = 1;
102     deg = _n - 1;
103     while (len < _n) len <= 1;
104     a = (ll*) calloc(len, sizeof(ll));
105     FOR (i, 0, deg) a[i] = f[i];
106 }
107 Poly derivative() {
108     Poly g(this->deg);
109     FOR (i, 1, this->deg) {
110         g.a[i - 1] = this->a[i] * i % mod;
111     }
112     return g;
113 }
114 Poly integral() {
115     Poly g(this->deg + 2);
116     FOR (i, 0, this->deg) {
117         g.a[i + 1] = this->a[i] * ::inv(i + 1) % mod;
118     }
119     return g;
120 }
121 Poly inv(int len1 = -1) {
122     if (len1 == -1) len1 = this->len;

```

```

123     Poly g(1); g.a[0] = ::inv(a[0]);
124     for (int l = 1; l < len1; l <= 1) {
125         Poly t; t = *this;
126         t.resize(l < 1);
127         t = g * g * t;
128         t.resize(l < 1);
129         Poly g1 = g * 2 - t;
130         swap(g, g1);
131     }
132     return g;
133 }
134 Poly ln(int len1 = -1) {
135     if (len1 == -1) len1 = this->len;
136     auto g = *this;
137     auto x = g.derivative() * g.inv(len1);
138     x.resize(len1);
139     x = x.integral();
140     x.resize(len1);
141     return x;
142 }
143 Poly exp() {
144     Poly g(1);
145     g.a[0] = 1;
146     for (int l = 1; l < len; l <= 1) {
147         Poly t, g1; t = *this;
148         t.resize(l < 1); t.a[0]++;
149         g1 = (t - g.ln(l < 1)) * g;
150         g1.resize(l < 1);
151         swap(g, g1);
152     }
153     return g;
154 }
155 Poly pow(ll n) {
156     Poly &a = *this;
157     int i = 0;
158     while (i <= a.deg and a.a[i] == 0) i++;
159     if (i and (n > a.deg or n * i > a.deg)) return Poly(a.deg + 1);
160     if (i == a.deg + 1) {
161         Poly res(a.deg + 1);
162         res.a[0] = 1;
163         return res;
164     }
165     Poly b(a.deg - i + 1);
166     int inv1 = ::inv(a.a[i]);
167     FOR (j, 0, b.deg)
168         b.a[j] = a.a[j + i] * inv1 % mod;
169     Poly res1 = (b.ln()) * (n % mod).exp() * (::power(a.a[i], n));
170     Poly res2(a.deg + 1);
171     FOR (j, 0, min((ll)(res1.deg), (ll)(a.deg - n * i)))
172         res2.a[j + n * i] = res1.a[j];
173     return res2;
174 }
175 };

```

7.16 Stirling's formula

$$n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$$

7.17 Theorem

1. $1 \sim x$ 質數的數量 $\approx \frac{x}{\ln x}$
2. x 的因數的數量 $\approx x^{\frac{1}{3}}$
3. x 的質因數的數量 $\approx \log \log x$

4. p is a prime number $\Leftrightarrow (p-1)! \equiv -1 \pmod{p}$

5. 每個正整數都可以表示成四個整數的平方和

6. 任何大於 2 的整數都可以表示成兩個質數的和

7. $n^{k-2} \cdot \prod_{i=1}^k s_i$ n 個點、 k 的連通塊，加上 $k-1$ 條邊使得變成一個連通圖的方法數，其中每個連通塊有 s_i 個點

7.18 josephus [0be067]

```

1 // n 個人，每 k 個人就刪除的約瑟夫遊戲
2 int josephus(int n, int k) {
3     if (n == 1)
4         return 0;
5     if (k == 1)
6         return n-1;
7     if (k > n)
8         return (josephus(n-1, k) + k) % n;
9     int cnt = n / k;
10    int res = josephus(n - cnt, k);
11    res -= n % k;
12    if (res < 0)
13        res += n;
14    else
15        res += res / (k - 1);
16    return res;
17 }

```

7.19 二元一次方程式

$$\begin{cases} ax + by = e \\ cx + dy = f \end{cases} = \begin{cases} x = \frac{ed-bf}{ad-bc} \\ y = \frac{af-ec}{ad-bc} \end{cases}$$

若 $x = \frac{0}{0}$ 且 $y = \frac{0}{0}$ ，則代表無限多組解。若 $x = \frac{*}{0}$ 且 $y = \frac{*}{0}$ ，則代表無解。

7.20 數論分塊 [8ccab5]

```

1 // 時間複雜度為  $O(\sqrt{n})$ 
2 // 區間為  $[L, r]$ 
3 for(int i=1; i<=n; i++){
4     int l = i, r = n/(n/i);
5     i = r;
6     ans.push_back(r);
7 }

```

7.21 最大質因數 [ca5e52]

```

1 void max_fac(int n, int &ret){
2     if (n<=ret || n<2) return;
3     if (isprime(n)){
4         ret = max(ret, n);
5         return;
6     }
7
8     int p = Pollard_Rho(n);
9     max_fac(p, ret), max_fac(n/p, ret);
10 }

```

7.22 歐拉公式 [7e25f3]

```

1 // phi(n) = 小於 n 並與 n 互質的正整數數量。
2 // O(sqrt(n)) · 回傳 phi(n)
3 int phi(int n){
4     int ret = n;
5
6     for (int i=2 ; i*i<=n ; i++){
7         if (n%i==0){
8             while (n%i==0) n /= i;
9             ret = ret*(i-1)/i;
10        }
11    }
12    if (n>1) ret = ret*(n-1)/n;
13
14    return ret;
15 }
16
17 // O(n) · 回傳 1~n 的 phi 值
18 vector<int> phi_1_to_n(int n) {
19     vector<int> primes, lpf(n + 1), phi(n + 1);
20     phi[0] = 0; phi[1] = 1;
21     for (int i = 2; i <= n; ++i) {
22         if (lpf[i] == 0) {
23             primes.push_back(i);
24             lpf[i] = i;
25         }
26         for (auto p : primes) {
27             if (i * p > n) break;
28             lpf[i * p] = p;
29             if (i % p == 0) break;
30         }
31         bool b = i % (lpf[i] * lpf[i]);
32         phi[i] = phi[i / lpf[i]] * (lpf[i] - b);
33     }
34     return phi;
35 }

```

7.23 歐拉定理

$$a^n \equiv \begin{cases} a^{n \bmod \varphi(m)}, & \gcd(a, m) = 1, \\ a^n, & \gcd(a, m) \neq 1, n < \varphi(m), \\ a^{(n \bmod \varphi(m)) + \varphi(m)}, & \gcd(a, m) \neq 1, n \geq \varphi(m). \end{cases} \pmod{m}$$

7.24 錯排公式

錯排公式：(n 個人中 · 每個人皆不再原來位置的組合數)

$$dp_i = \begin{cases} 1 & i = 0 \\ 0 & i = 1 \\ (i-1)(dp_{i-1} + dp_{i-2}) & \text{otherwise} \end{cases}$$

8 String

8.1 AC automation [018290]

```

1 struct ACAutomation{
2     vector<vector<int>> go;
3     vector<int> fail, match, pos;
4     int sz = 0; // 有效節點為 [0, sz] · 開陣列的時候要小心 !!!
5 }

```

```

6 ACAutomation(int n) : go(n, vector<int>(26)), fail(n),
7     match(n) {}
8
9 void add(string s){
10     int now = 0;
11     for (char c : s){
12         if (!go[now][c-'a']) go[now][c-'a'] = ++sz;
13         now = go[now][c-'a'];
14     }
15     pos.push_back(now);
16 }
17
18 void build(){
19     queue<int> que;
20     for (int i=0 ; i<26 ; i++){
21         if (go[0][i]) que.push(go[0][i]);
22     }
23     while (que.size()){
24         int u = que.front();
25         que.pop();
26         for (int i=0 ; i<26 ; i++){
27             if (go[u][i]){
28                 fail[go[u][i]] = go[fail[u]][i];
29                 que.push(go[u][i]);
30             }else go[u][i] = go[fail[u]][i];
31         }
32     }
33 }
34
35 // counting pattern
36 void buildMatch(string &s){
37     int now = 0;
38     for (char c : s){
39         now = go[now][c-'a'];
40         match[now]++;
41     }
42
43     vector<int> in(sz+1), que;
44     for (int i=1 ; i<=sz ; i++) in[fail[i]]++;
45     for (int i=1 ; i<=sz ; i++) if (in[i]==0) que.push_back(i);
46     for (int i=0 ; i<que.size() ; i++){
47         int now = que[i];
48         match[fail[now]] += match[now];
49         if (--in[fail[now]]==0) que.push_back(fail[now]);
50     }
51 }

```

8.2 Enumerate Runs [94ca46]

```

1 vector<array<int, 3>> enumerate_run(string s){
2
3     int n = s.size();
4     SuffixArray sa(s), saBar(string(s.rbegin(), s.rend()));
5     sa.init_lcp(), saBar.init_lcp();
6
7     set<pair<int, int>> ss;
8     vector<array<int, 3>> runs;
9
10    for (int len=1 ; len<=n ; len++){
11        vector<int> lcp;
12        for (int i=0 ; i+len<=n ; i+=len){
13            int pos1 = sa.pos[i];
14            int pos2 = saBar.pos[i+len];
15
16            lcp.push_back(sa.get_lcp(pos1, pos2));
17
18            for (int ll=0, rr=0 ; ll<lcp.size() ; rr++, ll=rr){
19                while (rr<lcp.size() && lcp[rr]>=len) rr++;
20
21                int preLen = 0;
22                if (ll!=0){
23                    int p = n-1;
24                    int pos1 = saBar.pos[p-(ll*len-1)];
25                    int pos2 = saBar.pos[p-((ll+1)*len-1)];
26                    preLen = saBar.get_lcp(pos1, pos2);
27                }
28                int sufLen = rr<lcp.size() ? lcp[rr] : 0;
29
30                int ansL = ll*len-preLen, ansR = (rr+1)*len-1+sufLen;
31                if (ansL!=ansR && ansR-ansL+1>=2*len && ss.find({ansL, ansR+1})==ss.end()){
32                    ss.insert({ansL, ansR+1});
33                    runs.push_back({len, ansL, ansR+1});
34                }
35            }
36        }
37        return runs;
38    }
39 }

```

```

15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39 }

```

8.3 Hash [942f42]

```

1 mt19937 seed(chrono::steady_clock::now().time_since_epoch().count());
2 int rng(int l, int r){
3     return uniform_int_distribution<int>(l, r)(seed);
4 }
5 int A = rng(1e5, 8e8);
6 const int B = 1e9+7;
7
8 // 2f6192
9 struct RollingHash{
10     vector<int> Pow, Pre;
11     RollingHash(string s = ""){
12         Pow.resize(s.size());
13         Pre.resize(s.size());
14
15         for (int i=0 ; i<s.size() ; i++){
16             if (i==0){
17                 Pow[i] = 1;
18                 Pre[i] = s[i];
19             }else{
20                 Pow[i] = Pow[i-1]*A%B;
21                 Pre[i] = (Pre[i-1]*A+s[i])%B;
22             }
23         }
24
25         return;
26     }
27
28     int get(int l, int r){ // 取得 [l, r] 的數值
29         if (l==0) return Pre[r];
30         int res = (Pre[r]-Pre[l-1]*Pow[r-l+1])%B;
31         if (res<0) res += B;
32         return res;
33     }
34 };

```

8.4 KMP [287490]

```

1 // KMP[i] = s[0...i] 的最長共同前後綴長度 · KMP[KMP[i]-1] 可以
  跳 fail Link
2 vector<int> KMP(string &s){
3     vector<int> ret(s.size());
4     for (int i=1 ; i<s.size() ; i++){
5         int j = ret[i-1];
6         while (j && s[i]!=s[j]) j = ret[j-1];
7         ret[i] = j + (s[i]==s[j]);
8     }
9     return ret;
10 }

```

8.5 Manacher [199269]

```

1 // 回傳所有最長 (無法往兩側擴展) 迴文的區間 [L, r], 0-based
2 vector<array<int, 2>> Manacher(string str) {
3     string tmp = "$#";
4     for(char i : str) {
5         tmp += i;
6         tmp += '#';
7     }
8
9     int m = tmp.size(), mx = 0, id = 0;
10    vector<int> p(m);
11    vector<array<int, 2>> res;
12    for(int i=1 ; i<m ; i++) {
13        p[i] = mx > i ? min(p[id*2-i], mx-i) : 1;
14        while(tmp[i+p[i]] == tmp[i-p[i]]) p[i]++;
15        if (mx < i + p[i]) mx = i + p[i], id = i;
16
17        if (p[i] > 1) {
18            int r = (i / 2) + (p[i] / 2) - 2 + (i & 1);
19            res.push_back({r - p[i] + 2, r});
20        }
21    }
22    return res;
23 }

```

8.6 Min Rotation [b24786]

```

1 int minRotation(string s) {
2     int a = 0, n = s.size();
3     s += s;
4
5     for (int b=0 ; b<n ; b++){
6         for (int k=0 ; k<n ; k++){
7             if (a+k==b || s[a+k]<s[b+k]){
8                 b += max(0LL, k-1);
9                 break;
10            }
11            if (s[a+k]>s[b+k]){
12                a = b;
13                break;
14            }
15        }
16    }
17    return a;
18 }

```

8.7 Suffix Array [47a062]

```

1 // 注意 · 當 |s|=1 時 · lcp 不會有值 · 務必測試 |s|=1 的 case
2 struct SuffixArray {
3     string s;
4     vector<int> sa, lcp;
5
6     // 0eea96
7     template<typename T>
8     void sais(const T s, int n, int *sa, int *lar, int *p,
9               int *t, int A) {
10        int *rnk = p + n, *q = p + n / 2, *bkt = lar + A;
11        int m = 0, i, j, x = t[n - 1] = 1, y = rnk[0] = -1,
12            cnt = -1;
13
14        for (i=n-2 ; ~i ; i--) t[i] = (s[i] == s[i + 1] ? t[i
15            + 1] : s[i] < s[i + 1]);
16        for (i=1 ; i<n ; i++) rnk[i] = t[i] && !t[i-1] ? (p[m
17            ]=i, m++) : -1;
18        fill_n(lar, A, 0);
19        for (i=0 ; i<n ; i++) ++lar[s[i]];
20        for (i=1 ; i<A ; i++) lar[i] += lar[i-1];
21        auto pushS = [&](int x) { sa[--bkt[s[x]]] = x; };
22        auto pushL = [&](int x) { sa[bkt[s[x]]++] = x; };
23        auto induce_sort = [&](int* v) {
24            fill_n(sa, n, 0);
25            copy_n(lar, A, bkt);
26            for (i=m-1 ; ~i ; i--) pushS(v[i]);
27            copy_n(lar, A - 1, bkt + 1);
28            for (i=0 ; i<n ; i++) if (sa[i] && !t[sa[i] - 1])
29                pushL(sa[i] - 1);
30            copy_n(lar, A, bkt);
31            for (i=n-1 ; ~i ; i--) if (sa[i] && t[sa[i] - 1])
32                pushS(sa[i] - 1);
33        };
34        induce_sort(p);
35        for (i=0 ; i<n ; i++){
36            if (~(x=rnk[sa[i]])){
37                j = y < 0 || memcmp(s + p[x], s + p[y], (p[x
38                    + 1] - p[x]) * sizeof(s[0]));
39                q[y = x] = cnt += j;
40            }
41        }
42        if (cnt+1<m) sais(q, m, sa, bkt, rnk, t + n, cnt + 1)
43            ;
44        else for (i=0 ; i<m ; i++) sa[q[i]] = i;
45        for (i=0 ; i<m ; i++) q[i] = p[sa[i]];
46        induce_sort(q);
47    }
48
49    // da9ddf, O(n + Lim), Lim = max{s[i]}
50    template<typename T>
51    SuffixArray(const T& _s) : s(_s.begin(), _s.end()) {
52        s.push_back(-1);
53        for (auto &i:s) i += 1;
54        int n = s.size(), lim = *max_element(s.begin(), s.end
55            ()) + 5;
56        sa.resize(n);
57        lcp.resize(n);
58        vector<int> bkt(n + lim * 2), p(n * 2), t(n * 2),
59            rank(n);
60        sais(&s[0], n, &sa[0], &bkt[0], &p[0], &t[0], lim);
61
62        for (int i=1 ; i<n ; i++) rank[sa[i]] = i;
63        for (int i=0, j, k=0 ; i<n-1 ; lcp[rank[i++]]=k){
64

```

```

65            for (k && k--, j=sa[rank[i]-1] ; i+k<s.size() &&
66                j+k<s.size() && s[i+k]==s[j+k] ; k++);
67        }
68        sa.erase(sa.begin());
69        lcp.erase(lcp.begin(), lcp.begin()+2);
70        s.pop_back();
71    }
72
73    // f49583
74    vector<int> pos; // pos[i] = i 這個值在 pos 的哪個地方
75    SparseTable st;
76    void init_lcp(){
77        pos.resize(sa.size());
78        for (int i=0 ; i<sa.size() ; i++){
79            pos[sa[i]] = i;
80        }
81        if (lcp.size()){
82            st.build(lcp);
83        }
84    }
85
86    // 用之前記得 init
87    // 查詢「sa 上的位置」的 x 跟 y 的 lcp
88    int get_lcp(int x, int y){
89        if (x==y) return s.size()-x;
90        if (x>y) swap(x, y);
91        return st.query(x, y);
92    }
93
94    // 回傳 [l1, r1] 跟 [l2, r2] 的 lcp · 0-based
95    int get_lcp(int l1, int r1, int l2, int r2){
96        int pos_1 = pos[l1], len_1 = r1-l1+1;
97        int pos_2 = pos[l2], len_2 = r2-l2+1;
98        if (pos_1>pos_2){
99            swap(pos_1, pos_2);
100            swap(len_1, len_2);
101        }
102        if (l1==l2){
103            return min(len_1, len_2);
104        }
105        else{
106            return min({st.query(pos_1, pos_2), len_1, len_2
107                });
108        }
109    }
110
111    // 檢查 [l1, r1] 跟 [l2, r2] 的大小關係 · 0-based
112    // 如果前者小於後者 · 就回傳 <0 · 相等就回傳 =0 · 否則回傳
113    >0
114    // 5b8db0
115    int substring_cmp(int l1, int r1, int l2, int r2){
116        int len_1 = r1-l1+1;
117        int len_2 = r2-l2+1;
118        int res = get_lcp(l1, r1, l2, r2);
119
120        if (res<len_1 && res<len_2){
121            return s[l1+res]-s[l2+res];
122        }
123        else if (len_1==res && len_2==res){
124            return 0;
125        }
126        else{
127            return len_1==res ? -1 : 1;
128        }
129    }

```

```

116 // 對於位置在 <=p 的後綴，找離他左邊/右邊最接近位置 >p 的
    後綴的 lcp · 0-based
117 // pre[i] = s[i] 離他左邊最接近位置 >p 的後綴的 lcp · 0-
    based
118 // suf[i] = s[i] 離他右邊最接近位置 >p 的後綴的 lcp · 0-
    based
119 // da12fa
120 pair<vector<int>, vector<int>> get_left_and_right_lcp(int
    p){
121     vector<int> pre(p+1);
122     vector<int> suf(p+1);
123
124     { // build pre
125         int now = 0;
126         for (int i=0 ; i<s.size() ; i++){
127             if (sa[i]<=p){
128                 pre[sa[i]] = now;
129                 if (i<lcp.size()) now = min(now, lcp[i]);
130             }else{
131                 if (i<lcp.size()) now = lcp[i];
132             }
133         }
134     }
135     { // build suf
136         int now = 0;
137         for (int i=s.size()-1 ; i>=0 ; i--){
138             if (sa[i]<=p){
139                 suf[sa[i]] = now;
140                 if (i-1>=0) now = min(now, lcp[i-1]);
141             }else{
142                 if (i-1>=0) now = lcp[i-1];
143             }
144         }
145     }
146
147     return {pre, suf};
148 }
149 };

```

8.8 Suffix Automata [efe54a]

```

1 // root node is 0
2 // node -> strings with the same endpos set,
3 //         length in range [len[fa] + 1, len]
4 // Link -> Longest suffix with different endpos set
5 // len -> longest path length from root
6 // cnt -> size of endpos set
7 // node's endpos set -> pos (1-based) in the subtree (Link)
  of node (Some node have pos = 0 and should be ignored)
8 struct SAM {
9     static constexpr int N = (int)(1e5 + 10), Z = 26;
10    int ch[2 * N][Z], len[2 * N], link[2 * N], pos[2 * N], cnt
        [2 * N], sz;
11    int newnode() {
12        fill_n(ch[sz], Z, 0);
13        len[sz] = link[sz] = pos[sz] = cnt[sz] = 0;
14        return sz++;
15    }
16    void build(string s) {
17        sz = 0, newnode(), link[0] = -1;
18        int lst = 0;
19        for (int i = 0; i < s.size(); ++i) {
20            int c = s[i] - 'a'; // you may need to change to 'A'
21            int cur = newnode();
22            len[cur] = len[lst] + 1, pos[cur] = i + 1;

```

```

23    int p = lst;
24    while (~p && !ch[p][c])
25        ch[p][c] = cur, p = link[p];
26    if (p == -1) link[cur] = 0;
27    else {
28        int q = ch[p][c];
29        if (len[p] + 1 == len[q]) {
30            link[cur] = q;
31        } else {
32            int nxt = newnode();
33            len[nxt] = len[p] + 1, link[nxt] = link[q];
34            pos[nxt] = 0;
35            for (int j = 0; j < Z; ++j)
36                ch[nxt][j] = ch[q][j];
37            while (~p && ch[p][c] == q)
38                ch[p][c] = nxt, p = link[p];
39            link[q] = link[cur] = nxt;
40        }
41    }
42    cnt[cur]++, lst = cur;
43 }
44 vector<int> p(sz);
45 iota(p.begin(), p.end(), 0);
46 sort(p.begin(), p.end(), [&](int i, int j) {
47     return len[i] > len[j];
48 });
49 for (int i = 0; i < sz; ++i) if (~link[p[i]])
50     cnt[link[p[i]]] += cnt[p[i]];
51 }
52 };

```

8.9 Wildcard Matching [e93074]

```

1 const int B = 27;
2 string p;
3 for (int i=0 ; i<B ; i++) p += ('a'+i);
4 p += '*';
5 vector<vector<double>> res(B);
6 for (int i=0 ; i<B ; i++){
7     vector<double> ss, tt;
8     for (auto x : s) ss.push_back(x==p[i] || x=='*');
9     for (auto x : t) tt.push_back(x==p[i] || x=='*');
10    reverse(tt.begin(), tt.end());
11    res[i] = PolyMul(ss, tt);
12 }
13 for (int i=t.size()-1 ; i+t.size()-1<res[0].size() ; i++){
14     int total = 0;
15     for (int j=0 ; j<B-1 ; j++) total += (int)abs(round(res[j
        ][i]));
16     total -= 25*(int)abs(round(res[26][i]));
17     cout << (total==t.size());
18 }

```

8.10 Z Algorithm [9d559a]

```

1 // z[i] 回傳 s[0...] 跟 s[i...] 的 lcp, z[0] = 0
2 vector<int> z_function(string s){
3     vector<int> z(s.size());
4     int l = -1, r = -1;
5     for (int i=1 ; i<s.size() ; i++){
6         z[i] = i>r ? 0 : min(r-i, z[i-1]);
7         while (i+z[i]<s.size() && s[i+z[i]]==s[z[i]]) z[i]++;
8         if (i+z[i]>r) l=i, r=i+z[i];
9     }
10    return z;

```